

EVENTIFY

(Smart Event Management and Ticket Booking System)

A Project Report

Submitted in partial fulfilment of the
Requirements for the award of degree of

BACHELOR IN INFORMATION TECHNOLOGY

BY

Name: Ibrahim Zummer

UID: 24BIT064

Under the esteemed guidance of
Mr. Wilson Rao and Ms. Bertilla Fernandes

DEPARTMENT OF SOFTWARE DEVELOPMENT

JAI HIND COLLEGE



(Empowered Autonomous)

MUMBAI, 400020

MAHARASHTRA

2026-27

JAI HIND COLLEGE

(Empowered Autonomous)

MUMBAI, 400020 MAHARASHTRA

DEPARTMENT OF SOFTWARE DEVELOPMENT



CERTIFICATE

This is to certify that the project entitled, “**EVENTIFY — Smart Event Management and Ticket Booking System**”, is the bonafide work of Ibrahim Zummer bearing UID (24BIT064), submitted in partial fulfilment of the requirements for the award of the degree of **BACHELOR IN INFORMATION TECHNOLOGY** from Jai Hind College (Empowered Autonomous), University of Mumbai.

The work presented in this report was carried out under our guidance and supervision during the academic year 2026-27, and has not been submitted elsewhere for the award of any other degree or diploma.

Internal Guide

Coordinator

Date

External Examiner

College Seal

DECLARATION

I hereby declare that the project entitled, “**EVENTIFY — Smart Event Management and Ticket Booking System**”, carried out at Jai Hind College, has not in any case been duplicated for submission to any other university for the award of any degree. To the best of my knowledge, other than myself, no one has submitted this work to any other university.

The project has been undertaken in partial fulfilment of the requirements for the award of the degree of **BACHELOR IN INFORMATION TECHNOLOGY**, and is submitted as the final semester project forming part of the prescribed curriculum.

All external material referred to during the course of this work — documentation, articles and technical references — has been acknowledged in the Bibliography, and all source code presented in this report was written by me except where explicitly stated otherwise.

Name and Signature of the Student

Ibrahim Zummer

ACKNOWLEDGEMENT

I would like to express my sincere thanks to everyone who supported me over the course of this project.

I am grateful to my internal guide, Ms. Bertilla Fernandes, for consistent direction and for the patience shown while reviewing successive drafts of both the application and this report. The suggestions offered at each review shaped decisions that materially improved the final system.

I owe my deep gratitude to Mr. Wilson Rao, Coordinator of the Department of Software Development, who took a keen interest in the progress of this work and guided me throughout its completion by providing all the information and resources necessary for developing a good system.

I would also like to thank the faculty of the Department of Software Development at Jai Hind College for the foundation laid across the programme, without which a project of this scope would not have been approachable.

At last, but not least, I want to thank all my friends who helped me in completing this project, with whom I exchanged ideas and thoughts that made it possible to complete the work accurately. I also wish to thank my parents for their personal support and encouragement throughout.

Ibrahim Zumner

ABSTRACT

Eventify is a web-based event management and ticket booking platform built to replace the spreadsheets, paper registers and scattered messaging that organisations still rely on to run seminars, workshops, cultural programmes and similar gatherings. The platform brings event publishing, ticket reservation, payment recording, attendee feedback and administrative oversight into a single application, so that a listing, the tickets sold against it and the money collected for it are all read from one source of truth rather than reconciled by hand after the fact.

The system is built on the MERN stack. A React 18 single-page application, bundled with Vite and styled with Bootstrap 5, provides the interface; an Express 5 server exposes a REST API; and MongoDB Atlas, accessed through the Mongoose ODM, stores the data. Authentication is stateless and token-based: passwords are hashed with bcrypt before storage, JSON Web Tokens carry the caller's identity and role, and every protected route passes through middleware that verifies the token and then checks the role against the operation being attempted.

Three characteristics distinguish the implementation. First, ticket inventory is treated as a resource that must be conserved — a booking decrements the available count through a single atomic conditional update, a cancellation restores it, and an event that sells out is returned to the published state automatically when stock comes back. Second, bookings and payments retain a denormalised snapshot of the event as it stood at the moment of purchase, so a subsequent edit to an event listing cannot silently rewrite the history of what a customer actually bought. Third, the platform supports two payment channels side by side: an instant demo gateway, and a manual UPI route in which the customer submits a transfer reference that an administrator reviews and confirms against a payment ledger before the ticket is activated. Confirmed bookings yield a ticket carrying a QR code, which the holder can export as a PDF, and attendees holding a confirmed booking may rate and review the event afterwards.

The application was verified through unit, integration and system testing across the authentication, event, booking, payment, ticketing, feedback and administrative modules, and deployed to a public cloud environment with the frontend on Vercel, the backend on Railway and the database on MongoDB Atlas. The result is a working demonstration that

the operational problems of manual event administration — duplicated records, uncertain ticket availability and untraceable payments — are addressable with a modest, well-structured full-stack application.

Keywords: *event management, online ticket booking, MERN stack, React, Express, MongoDB, JSON Web Token, role-based access control, REST API, QR ticketing.*

INDEX

Sr No			PARTICULARS	PAGE NO.	DATE
1			INTRODUCTION	1-6	
	1.1		Background	1	
	1.2		Objectives	2	
	1.3		Purpose, Scope and Applicability	2	
		1.3.1	Purpose	2	
		1.3.2	Scope	3	
		1.3.3	Applicability	3	
	1.4		Achievements	4	
	1.5		Organisation of Report	5	
2			SURVEY OF TECHNOLOGIES	7-12	
	2.1		Introduction	7	
	2.2		Front-End Technologies	7	
	2.3		Back-End Technologies	9	
	2.4		Database Technology	10	
	2.5		Development and Deployment Tools	11	
	2.6		Summary	11	
3			REQUIREMENT AND ANALYSIS	13-34	
	3.1		Problem Definition	13	
	3.2		Requirements Specification	14	
	3.3		Planning and Scheduling	16	
	3.4		Software and Hardware Requirements	18	
	3.5		Preliminary Product Description	19	
	3.6		Conceptual Models	21	
		3.6.1	Event Table	21	
		3.6.2	Entity Relationship Diagram	22	
		3.6.3	Class Diagram	25	
		3.6.4	Object Diagram	27	
		3.6.5	Use Case Diagram	27	
		3.6.6	Activity Diagram	28	
		3.6.7	Sequence Diagram	29	
		3.6.8	Statechart Diagram	30	
		3.6.9	Package Diagram	32	
		3.6.10	Deployment Diagram	32	
		3.6.11	Data Flow Diagram	33	

Sr No			PARTICULARS	PAGE NO.	DATE
4			SYSTEM DESIGN	35-50	
	4.1		Basic Modules	35	
	4.2		Data Design	39	
		4.2.1	Schema Design	39	
		4.2.2	Data Integrity and Constraints	43	
	4.3		User Interface Design	44	
	4.4		Security Issues	46	
	4.5		Test Cases Design	48	
5			IMPLEMENTATION AND TESTING	51-64	
	5.1		Implementation Approach	51	
	5.2		Coding Details and Code Efficiency	53	
	5.3		Testing Approach	60	
		5.3.1	Unit Testing	61	
		5.3.2	Integration Testing	61	
		5.3.3	System Testing	62	
	5.4		Modifications and Improvements	63	
6			RESULTS AND DISCUSSION	65-70	
	6.1		Test Reports	65	
	6.2		User Documentation	67	
7			CONCLUSION	71-76	
	7.1		Conclusion	71	
	7.2		Limitations of the System	73	
	7.3		Future Scope of the Project	74	
			REFERENCES	77-79	
			GLOSSARY	80-81	

INTRODUCTION

1.1 Background

Events are the visible working life of an institution. Seminars, workshops, technical competitions, cultural festivals, sports meets and departmental programmes are how a college or an organisation assembles people around a purpose, and each of them carries an administrative tail: someone has to announce it, someone has to collect the names of those attending, someone has to take the money where money is involved, and someone has to check arrivals on the day.

For most organisations that tail is still handled by hand. A notice announces the event. A spreadsheet, a paper register or a messaging group collects registrations. Payments are taken separately and recorded by whoever collected them. On the day, arrivals are checked against whichever list the person at the door happens to be holding. Every one of those steps works in isolation; the difficulty is that they do not connect to one another, and the work of connecting them falls on people.

Web technology has changed what is possible here. A browser-based platform can hold the event listing, the registrations against it and the payments received for it as parts of one stored record rather than as three separate artefacts that a person is expected to keep aligned. Ticket availability can be decremented by the same operation that creates a booking, so that the number published to a prospective attendee cannot drift from reality. Event details can be published once and read everywhere, so that a change of venue propagates by itself. And every payment can be written to an auditable ledger tied to the booking it settles.

Eventify — the Smart Event Management and Ticket Booking System described in this report — is a web-based application built on that principle. It allows administrators and registered users to publish and maintain events, allows attendees to browse those events, reserve tickets, settle payment and hold a QR-coded ticket, and gives administrators oversight of every event, booking, payment, review and account on the platform. It replaces manual record keeping with a single, secure, centrally stored source of truth, and in doing so removes the reconciliation work rather than merely making it easier.

1.2 Objectives

The objectives of the Smart Event Management and Ticket Booking System are as follows:

- To develop a secure, centralised, web-based platform for managing events online.
- To provide secure user registration and authentication, with a separate and protected administrative sign-in.
- To enable events to be created, updated, withdrawn and managed across a defined lifecycle of draft, published, sold-out and cancelled states.
- To allow users to browse published events, view their full details and reserve tickets online.
- To keep ticket availability consistent with the set of active bookings at all times, so that overselling is structurally prevented rather than merely discouraged.
- To record every payment attempt in an auditable ledger tied to the booking it settles.
- To issue a QR-coded ticket for every confirmed booking and allow the holder to export it as a PDF document.
- To provide an online feedback and rating facility for users who have attended an event.
- To give administrators consolidated oversight of all events, bookings, payments, reviews and registered accounts.
- To reduce the paperwork and manual effort involved in event administration, and to improve the efficiency, reliability and accessibility of event information.

1.3 Purpose, Scope and Applicability

1.3.1 Purpose

The purpose of this project is to develop a web-based platform that simplifies the management of events and the booking of tickets against them. The system allows administrators and event owners to maintain event information, while enabling users to register, browse events, book tickets, settle payment, obtain a ticket and submit feedback after attending. It serves as a centralised solution that reduces paperwork, improves operational efficiency and removes the manual reconciliation that characterises the process it replaces.

1.3.2 Scope

The scope of the Smart Event Management and Ticket Booking System includes:

- User registration and authentication, with role-based separation of users and administrators.
- Event creation, modification, withdrawal and lifecycle management.
- Public event discovery, including a featured selection on the landing page and individual event pages addressed by a URL-friendly slug.
- Online ticket booking validated against live availability, with automatic decrement of the available count and automatic transition to a sold-out state.
- Booking history, booking cancellation and restoration of released tickets to the event.
- Payment settlement through three channels — free, demo gateway and manually reviewed UPI transfer — with every attempt written to a payment ledger.
- QR-coded ticket generation and PDF export for confirmed bookings.
- Event feedback and star ratings submitted by attendees holding a confirmed booking.
- Administrative consoles for events, bookings, payments, feedback and registered users.
- Secure storage and retrieval of application data on a cloud-hosted MongoDB Atlas cluster.
- A responsive interface, with light and dark themes, usable on desktop, tablet and mobile browsers.

The system is intended for educational institutions, workshops, seminars, conferences, cultural programmes and comparable event management activities. It does not cover live payment gateway settlement, ticket scanning at a venue entrance, or notification by email or SMS; these are recorded as limitations in Section 7.2 and as future work in Section 7.3.

1.3.3 Applicability

The system can be adopted by any organisation that runs events with any regularity. It is applicable to:

- Colleges and universities managing academic, technical and cultural programmes.
- Student societies and departmental committees running their own events within a larger institution.
- Corporate organisations conducting conferences, meetings and training programmes.

- Event management companies organising public and private events.
- Non-governmental organisations conducting awareness programmes and workshops.
- Community organisations, clubs and cultural associations managing social and recreational events.

One structural characteristic is worth noting here, because it widens the applicability of the platform. Event publication is not restricted to administrators: any registered user may publish an event, and administrators moderate rather than originate. This suits a setting — a college in which individual societies each run their own programme, for instance — where a single administrative office would be a bottleneck.

1.4 Achievements

The following were achieved over the course of developing the Smart Event Management and Ticket Booking System:

- A complete, centralised, web-based event management and ticket booking system was developed and deployed.
- Secure user registration and login were implemented, with passwords hashed using bcrypt before storage.
- Stateless authentication was implemented using JSON Web Tokens, with separate administrator authentication and role-based access control enforced as route middleware.
- A complete set of REST APIs was developed using Node.js and Express 5, covering authentication, events, bookings, payments, feedback and system health.
- Ticket inventory was made a conserved quantity: an atomic conditional update decrements availability as part of the same database operation that validates it, and cancellation restores both the count and the event's published status.
- A payment subsystem was implemented with three channels — free, demo gateway and manually reviewed UPI transfer — all writing to a common auditable ledger.
- QR-coded tickets were implemented, with client-side PDF export, using libraries loaded on demand so that the initial page payload is unaffected.
- A feedback and rating module was implemented, restricted to attendees holding a confirmed booking and enforcing one review per user per event.

- Administrative consoles were built for events, bookings, payments, feedback and users, including the ability to create, cancel and transfer a booking on a user's behalf.
- Application data was stored on a cloud-hosted MongoDB Atlas cluster, accessed through Mongoose schemas that enforce field-level validation.
- Protected routing was implemented on both the client and the API, so that access control is enforced at the endpoint rather than assumed from the interface.
- Unit, integration and system testing were carried out across all modules, and the defects found were resolved and recorded.
- The application was deployed to a public cloud environment, with the frontend on Vercel, the backend on Railway and the database on MongoDB Atlas.
- A responsive, accessible interface with light and dark themes was designed and implemented using Bootstrap 5.

1.5 Organisation of Report

This report is organised into seven chapters, each covering a distinct stage in the development of the system.

Chapter 1 — Introduction. Sets out the background to the project, its objectives, its purpose, scope and applicability, the achievements realised, and the organisation of this report.

Chapter 2 — Survey of Technologies. Describes the technologies, frameworks, libraries, tools and hosting platforms used to build the system, and explains why each was selected.

Chapter 3 — Requirement and Analysis. Defines the problem the system addresses, specifies its functional and non-functional requirements, presents the project plan and schedule, states the hardware and software requirements, gives a preliminary product description, and presents the conceptual models — the event table and the full set of UML, entity-relationship and data-flow diagrams.

Chapter 4 — System Design. Presents the modular decomposition of the system, the database schema design and the constraints that maintain its integrity, the user interface design, the security measures applied at each layer, and the design of the test cases.

Chapter 5 — Implementation and Testing. Explains the implementation strategy and architecture, presents and discusses the code behind the system's central mechanisms,

describes the measures taken for code efficiency, sets out the testing approach across unit, integration and system levels, and records the modifications made in response to what testing found.

Chapter 6 — Results and Discussion. Reports the results of executing the designed test cases, analyses the defects found and how they were resolved, and provides the user documentation for both roles the system recognises.

Chapter 7 — Conclusion. Draws conclusions about the completed system and its significance, states its limitations plainly, and sets out the directions in which it could be extended.

SURVEY OF TECHNOLOGIES

2.1 Introduction

The Smart Event Management and Ticket Booking System is built on the MERN stack — MongoDB, Express, React and Node.js — a combination in which a single language, JavaScript, is used across the interface, the server and the database query layer. This chapter surveys each technology used in the project, describes the role it plays in the system, and gives the reason it was selected over the alternatives available.

The selection was governed by three considerations. The system had to be reachable from any modern browser without installation, which pointed to a single-page web application. It had to store data whose shape differs meaningfully between entity types — an event, a booking and a payment share little structure — which suited a document database over a strictly relational one. And it had to be developed and deployed within an academic timeframe by one developer, which favoured a stack with a single language, mature tooling and free hosting tiers.

2.2 Front-End Technologies

2.2.1 React 18

React is an open-source JavaScript library, developed and maintained by Meta, for building user interfaces from composable components. It maintains a virtual representation of the document and reconciles it against the real one, so that only the parts of a page whose data has actually changed are re-rendered.

In this system React provides the entire presentation tier. Interface state is held in function components using hooks, and the two pieces of state that cross the whole application — the authenticated session and the selected theme — are provided through React's Context API rather than passed down manually through every intervening component.

Advantages: component-based architecture that makes interface elements reusable; efficient rendering through virtual DOM reconciliation; a declarative model in which the interface is described as a function of state; first-class support for single-page routing; and an exceptionally large ecosystem and community.

2.2.2 Vite

Vite is the build tool and development server for the frontend. During development it serves source files as native ES modules, so the server starts almost instantly regardless of project size and a saved change is reflected in the browser through hot module replacement without a full reload. For production it bundles the application with Rollup into a small set of optimised static assets. Vite also provides the development proxy that forwards API calls to the backend, which avoids cross-origin complications during local development.

2.2.3 React Router DOM

React Router provides client-side routing, mapping browser URLs to components without a round trip to the server. The project uses its nested-route model: layout routes supply the shared page structure, and guard routes wrap the protected areas so that authentication and role checks are declared once in the route table rather than repeated inside each page.

2.2.4 Bootstrap 5

Bootstrap is the CSS framework used for layout and interface components. Its twelve-column responsive grid, mobile-first breakpoints and pre-built components — navigation bars, cards, forms, tables, modals, alerts and badges — provide a consistent visual language without hand-writing a complete stylesheet. Version 5 requires no jQuery dependency, which keeps the bundle small. Project-specific styling, including the light and dark theme definitions, is layered above it in a single global stylesheet using CSS custom properties.

2.2.5 HTML5 and CSS3

HTML5 provides the semantic structure of the rendered document, and CSS3 supplies the styling layer above Bootstrap — custom properties for theming, flexbox and grid for layout, media queries for responsiveness, and transitions for interface feedback.

2.2.6 Axios

Axios is the HTTP client through which the frontend communicates with the API. A single configured instance carries the base URL and request timeout, and a request interceptor attaches the stored access token to the Authorization header of every outgoing

call. Centralising this means no individual page has to remember to authenticate its own requests.

2.2.7 qrcode, jsPDF and html2canvas

These three libraries produce the ticket. The qrcode library encodes the booking's identifying details into a scannable QR code rendered as a data URL; html2canvas rasterises the rendered ticket element to a canvas; and jsPDF places that canvas into a PDF document sized to fit it. All three are loaded through dynamic imports at the moment a ticket is requested, so the majority of visitors — who never download a ticket — never fetch them.

2.3 Back-End Technologies

2.3.1 Node.js

Node.js is the server-side JavaScript runtime on which the API executes. Its event-driven, non-blocking I/O model suits an application whose work is dominated by waiting on database queries rather than by computation, since a single process can hold many concurrent requests in flight. Using JavaScript on the server also means one language across the whole stack, which removes a context switch for a single developer.

2.3.2 Express 5

Express is the web framework layered over Node's HTTP module. It supplies routing and a middleware pipeline through which every request passes. The middleware model is what makes the project's security design possible: authentication and authorisation are written once as middleware functions and then composed into each route declaration, so a route's protection is visible in the route table itself. Version 5 additionally forwards rejected promises from async handlers to the error middleware automatically, removing the try/catch boilerplate earlier versions required.

2.3.3 Mongoose

Mongoose is the Object Data Modelling library that sits between the application and MongoDB. It adds a schema layer above MongoDB's otherwise schema-less collections, declaring which fields a document may contain, their types, whether they are required, and what values they may hold. It also provides middleware hooks — the password-

hashing hook on the User schema is one — and the `populate()` mechanism for resolving references between documents.

2.3.4 JSON Web Token

The `jsonwebtoken` library issues and verifies the access tokens that carry identity. A token is signed with a secret held in environment configuration and carries the account identifier and role in its payload. Because the token is self-describing and signature-verified, the server holds no session state, which is what allows the API to be scaled horizontally without shared session storage.

2.3.5 bcrypt

`bcrypt` hashes passwords before they are stored. It is a deliberately slow, adaptive hash function: a work factor — ten salt rounds in this system — controls the computational cost of each hash, so that brute-force guessing remains expensive even as hardware improves. Each hash incorporates a distinct random salt, so two accounts sharing a password produce different stored values and precomputed lookup tables are useless against them.

2.3.6 Supporting Middleware

- `cors` — restricts browser access to the API to the configured frontend origin.
- `dotenv` — loads configuration secrets from an environment file kept out of version control.
- `morgan` — logs incoming HTTP requests during development.
- `nodemon` — restarts the development server automatically when a source file changes.

2.4 Database Technology

2.4.1 MongoDB and MongoDB Atlas

MongoDB is a document-oriented NoSQL database that stores records as BSON documents in collections. Because documents within a collection need not share an identical structure, the database accommodates entities of genuinely different shape — a

user, an event, a booking and a payment — without the table-per-entity rigidity of a relational schema, and without a schema migration each time a field is added.

MongoDB Atlas is the managed cloud service on which the database is hosted. It supplies provisioning, automated backup, monitoring and network access control without server administration, and its free tier is sufficient for a project of this scale. The application connects to it over an authenticated connection string held in environment configuration.

Why a document database here. The dominant access pattern in this system is to read one whole entity at a time — an event with all its fields, a booking with its event snapshot — rather than to compose a result from several normalised tables. A document store serves that pattern in a single read. Where relational guarantees are genuinely needed, Mongoose supplies them at the schema level: required fields, uniqueness constraints, enumerated values, and minimum and maximum bounds.

2.5 Development and Deployment Tools

Tool	Role in the project
Visual Studio Code	Source code editor, with IntelliSense, integrated terminal, debugger and Git integration.
Git	Distributed version control; the commit history records the project's development sequence and is the basis of the schedule in Section 3.3.
GitHub	Remote repository hosting, backup and the source from which the deployment platforms build.
Postman	REST API testing. Every endpoint was exercised here — with valid input, invalid input, a missing token and a wrong role — before any interface was built against it.
StarUML	UML, entity-relationship and data-flow modelling; the source of the diagrams in Section 3.6.
Vercel	Hosting for the frontend. Serves the built static bundle over HTTPS from a content delivery network, with a rewrite rule directing all paths to index.html so that client-side routing resolves correctly on a refresh.
Railway	Hosting for the Express API, with environment variables supplied through the platform rather than committed to the repository.
MongoDB Atlas	Managed cloud hosting for the database cluster.

2.6 Summary

The system uses React 18 with Vite and Bootstrap 5 for the presentation tier, Express 5 on Node.js for the application tier, and MongoDB Atlas accessed through Mongoose for the data tier, with JSON Web Tokens and bcrypt providing authentication and credential

security. The three tiers are deployed independently — the frontend on Vercel, the backend on Railway and the database on MongoDB Atlas — and communicate over HTTPS using JSON. Together these technologies provide a secure, scalable and maintainable foundation that can be developed and operated by a single developer, and that accommodates each of the extensions described in Section 7.3 without structural change.

REQUIREMENT AND ANALYSIS

3.1 Problem Definition

Event administration in most institutions is not a single process but a loose collection of habits. A notice announces the event. A spreadsheet, a paper register or a messaging group collects the names of those who want to attend. Money, where money is involved, changes hands separately and is recorded by whoever collected it. On the day, someone checks arrivals against whichever list they happen to be holding. Each of these steps works in isolation. The difficulty is that they do not connect, and the work of connecting them falls on people.

Four specific failures follow from that disconnection.

Records fragment and disagree. Because registration, payment and attendance are recorded separately, reconciling them is a manual comparison performed after the fact. When the lists disagree — a name on the register with no matching payment, a payment with no matching name — there is often no way to establish which is correct, because neither record carries enough context to settle the question.

Availability is unreliable. The number of places remaining is a figure someone maintains by hand. Between the moment a registration is accepted and the moment the count is updated, the published availability is wrong. When two registrations arrive close together, or when several people are accepting registrations at once, the count drifts, and overselling becomes a routine occurrence rather than an exception.

Information decays. A change to a venue, a start time or a ticket price has to be propagated by hand to every place it was published. Whichever copy is not updated becomes a source of incorrect information that continues to circulate, and attendees who acted on it arrive at the wrong place or the wrong time.

Payment is untraceable. A payment collected in cash or by direct transfer leaves no structured record tying it to a specific person and a specific event. Verifying that a given attendee has paid becomes an exercise in memory or in searching through transaction messages, and refunding a cancellation is correspondingly awkward.

These are not failures of care. They are structural consequences of holding related information in unrelated places. Solving them requires a system in which the registration,

the ticket count and the payment are parts of one stored record rather than three separate artefacts that a person is expected to keep aligned.

The problem this project addresses is therefore the design and implementation of a web-based platform in which an event, the tickets sold against it and the payments received for it are structurally connected; in which ticket availability is decremented by the same operation that creates a booking, so that it cannot drift; in which event details are published once and read everywhere; and in which every payment attempt is written to an auditable ledger tied to the booking it settles.

3.2 Requirements Specification

3.2.1 Functional Requirements

The functional requirements describe what the system is required to do. They are grouped by the module responsible for them.

ID	Module	Requirement
FR-01	User Management	The system shall allow a visitor to register an account by supplying a name, an email address and a password.
FR-02	User Management	The system shall reject a registration whose email address is already associated with an existing account.
FR-03	User Management	The system shall authenticate a registered user against a stored password hash and issue a signed access token on success.
FR-04	User Management	The system shall provide a separate authentication route for administrators, rejecting non-administrative accounts.
FR-05	User Management	The system shall return the profile of the currently authenticated account, and shall allow that account to update its profile image.
FR-06	Event Management	The system shall allow an authenticated user to create an event with a title, description, category, venue, city, organiser name, banner image, start and end date-time, ticket price and total ticket count.
FR-07	Event Management	The system shall generate a unique URL-friendly slug for each event from its title.
FR-08	Event Management	The system shall allow the owner of an event, and any administrator, to update or delete that event.
FR-09	Event Management	The system shall publish a list of events whose status is published or sold out, and shall serve an individual event by its slug.
FR-10	Event Management	The system shall allow an administrator to designate an event as featured for a bounded duration, and to withdraw that designation.
FR-11	Booking Management	The system shall allow an authenticated user to book one or more tickets against a published event.
FR-12	Booking Management	The system shall reject a booking whose requested quantity exceeds the tickets currently available.
FR-13	Booking Management	The system shall decrement the event's available ticket count when a booking is created, and set the event's status to sold out when the count reaches zero.
FR-14	Booking Management	The system shall assign each booking a unique human-readable

ID	Module	Requirement
		reference code.
FR-15	Booking Management	The system shall present each user with a history of their own bookings, and shall allow a user to cancel a booking.
FR-16	Booking Management	The system shall restore the cancelled quantity to the event's available ticket count on cancellation, returning a sold-out event to the published state.
FR-17	Payment Management	The system shall settle a zero-priced booking without requiring payment.
FR-18	Payment Management	The system shall provide a demo gateway path that confirms a paid booking immediately.
FR-19	Payment Management	The system shall provide a manual UPI path in which the user submits a transfer reference for administrative review.
FR-20	Payment Management	The system shall allow an administrator to confirm a reviewed UPI payment, which activates the associated ticket.
FR-21	Payment Management	The system shall record every payment attempt in a payment ledger with a unique reference, provider, method, amount, currency and status.
FR-22	Ticketing	The system shall generate a QR code for each confirmed booking and allow the holder to export the ticket as a PDF document.
FR-23	Feedback Management	The system shall allow a user holding a confirmed booking to submit a star rating and a written review for that event, and to edit or withdraw it afterwards.
FR-24	Feedback Management	The system shall permit at most one review per user per event, and shall display the aggregate rating and the reviews on the event page.
FR-25	Administration	The system shall provide administrators with consolidated listings of all events, bookings, payments, reviews and registered users.
FR-26	Administration	The system shall allow an administrator to create, cancel and transfer a booking on a user's behalf, and to remove a review.
FR-27	Administration	The system shall expose a health endpoint reporting API and database availability.

3.2.2 Non-Functional Requirements

The non-functional requirements describe the quality attributes the system must exhibit.

ID	Attribute	Requirement
NFR-01	Security	Passwords shall never be stored or transmitted in plain text; they shall be persisted only as bcrypt hashes.
NFR-02	Security	Every endpoint that reads or modifies account-specific data shall require a valid access token, and shall additionally verify the caller's role where the operation is role-restricted.
NFR-03	Security	Configuration secrets — the database connection string, the token signing secret and the administrator bootstrap credentials — shall be supplied through environment variables and excluded from version control.
NFR-04	Security	The API shall accept browser requests only from the configured frontend origin.
NFR-05	Performance	Interactive pages shall render within approximately two seconds on a broadband connection, and libraries used only for ticket export shall be loaded on demand rather than in the initial bundle.

ID	Attribute	Requirement
NFR-06	Reliability	Ticket availability shall remain consistent with the set of active bookings at all times, including after cancellations and under concurrent requests.
NFR-07	Reliability	Historical bookings and payments shall continue to display the event details in force at the time of purchase, irrespective of later edits to the event.
NFR-08	Usability	The interface shall be operable on desktop, tablet and mobile viewport widths without horizontal scrolling, and shall offer a light and a dark theme.
NFR-09	Usability	Every rejected input shall produce a specific, human-readable message identifying what was wrong.
NFR-10	Maintainability	The codebase shall separate routing, request handling, data modelling and cross-cutting middleware into distinct layers.
NFR-11	Portability	The application shall run on any platform supporting a current Node.js runtime and a modern browser, with no client-side installation required.
NFR-12	Scalability	The API shall hold no server-side session state, so that additional instances can serve requests without shared session storage.

3.3 Planning and Scheduling

Development was planned in two complementary views. The PERT network expresses the logical plan — which activities depend on which, and where the schedule has no slack. The Gantt chart expresses the calendar plan actually followed, in which several activities were deliberately overlapped rather than run strictly in sequence.

3.3.1 Activity Breakdown and PERT Analysis

ID	Activity	Duration (days)	Depends on
A	Requirement Gathering	3	—
B	System Analysis	2	A
C	Database Design	3	B
D	User Interface Design	4	B
E	Frontend Development	8	D
F	Backend Development	8	C
G	Integration	4	E, F
H	Testing	4	G
I	Deployment and Documentation	3	H

Applying the forward and backward pass to this network gives a project duration of 28 days along the critical path $A \rightarrow B \rightarrow D \rightarrow E \rightarrow G \rightarrow H \rightarrow I$. Database Design (C) and Backend Development (F) each carry one day of slack: the design work on the data model

could begin, and the corresponding backend work could finish, one day later than planned without delaying integration. Every other activity is critical, which is the expected shape for a single-developer project — with no second developer to absorb work in parallel, almost every task lies directly on the path.

The PERT sequence is therefore:

$$A \rightarrow B \rightarrow (C, D) \rightarrow (E, F) \rightarrow G \rightarrow H \rightarrow I$$

3.3.2 Project Schedule

The calendar schedule below covers the development window, with three deliberate overlaps. Frontend development began before system and database design was complete, because the interface work for authentication and event browsing did not depend on the final shape of the payment schema. Backend development ran concurrently with frontend development, with the API for each feature built immediately before the interface that consumed it. Testing began while payment integration was still in progress, since the authentication, event and booking modules were by then stable enough to test in earnest.

Phase	Activities	Duration
Phase 1	Requirements gathering, problem identification, project planning and feasibility study	3 days
Phase 2	System analysis; existing and proposed system comparison	2 days
Phase 3	Database and schema design; entity-relationship and class modelling	3 days
Phase 4	User interface design; wireframes, layouts and theme definition	4 days
Phase 5	Frontend development — pages, components, routing, context and services	8 days
Phase 6	Backend development — models, controllers, routes, middleware, authentication and the payment ledger	8 days
Phase 7	Integration of frontend, backend and MongoDB Atlas; ticketing and feedback modules	4 days
Phase 8	Unit, integration and system testing; defect resolution	4 days
Phase 9	Deployment to Vercel and Railway; documentation	3 days
	TOTAL DURATION	28 days

3.4 Software and Hardware Requirements

3.4.1 Hardware Requirements

Component	Minimum	Recommended
Processor	Intel Core i3 (8th generation) or AMD equivalent	Intel Core i5/i7, AMD Ryzen 5 or Apple M-series
Memory	4 GB RAM	8 GB RAM or higher
Storage	20 GB free disk space	40 GB free on solid-state storage
Display	1366 × 768	1920 × 1080
Network	Broadband connection	Stable broadband connection

These figures describe a development machine. The deployed application imposes no such requirement on its users: because the frontend is a static bundle served over HTTPS and the API runs on a remote host, an end user needs only a current web browser and a network connection, on a desktop computer, tablet or phone.

3.4.2 Software Requirements

Software	Version	Purpose
Windows 10/11, macOS or Linux	—	Development operating system
Node.js	18 LTS or later	JavaScript runtime for the API and build tooling
npm	9 or later	Dependency management
Visual Studio Code	Current	Source code editor
Git	2.x	Version control
React	18.3	Frontend user interface library
Vite	7.1	Frontend build tool and development server
React Router DOM	6.30	Client-side routing
Bootstrap	5.3	Responsive CSS framework
Axios	1.11	HTTP client
qrcode / jsPDF / html2canvas	1.5 / 4.2 / 1.4	QR encoding and PDF ticket export
Express	5.1	Backend web framework
Mongoose	8.18	Object data modelling for MongoDB
jsonwebtoken	9.0	Access token issue and verification
bcrypt	6.0	Password hashing
cors / dotenv / morgan	2.8 / 17.2 / 1.10	Origin control, configuration, request logging
nodemon	3.1	Development server auto-restart
MongoDB Atlas	Cloud (M0)	Managed database cluster
Postman	Current	REST API testing

Software	Version	Purpose
StarUML	Current	UML, ER and data-flow modelling
Google Chrome / Microsoft Edge	Current	Target browsers
Vercel / Railway	Cloud	Frontend and backend hosting

3.5 Preliminary Product Description

3.5.1 Overview

Eventify is a web application that brings event publication, ticket booking, payment recording, attendee feedback and administrative oversight into a single platform. A registered user may publish an event, browse events published by others, reserve tickets against them, settle the associated payment through one of the supported channels, hold the resulting QR-coded ticket, and review the event after attending. An administrator has oversight of the whole platform: every event, every booking, every payment, every review and every registered account.

The application is reached through a web browser and requires no installation. It presents a public face — a landing page with featured events, an events listing and individual event pages — that is available without an account, and an authenticated area whose contents depend on the role of the account signed in.

3.5.2 Product Features

- Account registration and authentication, with a separate administrative sign-in reached through a hidden trigger.
- Event creation, editing, withdrawal and lifecycle management across the draft, published, sold-out and cancelled states.
- Public event discovery with a time-bounded featured selection and individual event pages addressed by slug.
- Ticket booking validated against live availability, with atomic decrement of the available count and automatic transition to sold out.
- Booking history, booking cancellation and automatic restoration of released inventory.

- Three payment channels — free, demo gateway and manually reviewed UPI transfer — all writing to a common auditable ledger.
- QR-coded tickets with client-side PDF export.
- Star ratings and written reviews from attendees holding a confirmed booking, aggregated onto the event page.
- Administrative consoles for events, bookings, payments, feedback and users, including booking creation, cancellation and transfer on a user's behalf.
- A responsive interface with light and dark themes.
- A health endpoint reporting API and database availability.

3.5.3 User Characteristics

The system recognises two roles.

Registered User. May register and sign in; browse and search published events; view event details; book tickets; settle payment through the available channels; view their booking history; download a QR-coded ticket as a PDF; cancel a booking; publish and manage their own events; submit, edit and withdraw a review for an event they attended; and manage their profile image and theme preference.

Administrator. Signs in through a separate, unlinked route; has oversight of every event, booking, payment, review and account on the platform; may edit or delete any event and control featured placement; may confirm reviewed UPI payments; may create, cancel or transfer a booking on a user's behalf; may remove a review; and may query system health.

3.5.4 Product Perspective

The system is a self-contained three-tier web application. The presentation tier is a React single-page application served as a static bundle. The application tier is an Express REST API. The data tier is a MongoDB Atlas cluster reached only by the application tier. The tiers exchange JSON over HTTPS, and because the API holds no session state, identity travels in a signed token attached to each request. The three tiers are deployed independently, so each can be scaled or replaced without disturbing the others.

3.5.5 Benefits of the Proposed System

- Removes the manual reconciliation of registrations, ticket counts and payments by holding them as one connected record.
- Prevents overselling structurally rather than by procedure.
- Keeps historical bookings truthful even after the underlying event is edited.
- Provides a complete, auditable trail of every payment attempt and its outcome.
- Distributes event publication across users rather than routing every listing through a single administrative office.
- Provides real-time access to event information from any device with a browser.
- Produces a durable, verifiable ticket artefact for every confirmed booking.
- Gives organisers structured feedback from attendees rather than informal impressions.

3.5.6 Constraints

- The application requires an internet connection; there is no offline mode.
- Only registered users may book tickets or publish events.
- Payment is settled through a demo gateway or a manually reviewed UPI transfer rather than through a live commercial gateway.
- Tickets are counted rather than assigned; venues with numbered seating cannot be represented.
- Banner images are stored inside event documents rather than in object storage, which bounds the practical size of the image catalogue.

3.6 Conceptual Models

The conceptual models below describe the system independently of its implementation. The event table records what the system responds to; the entity-relationship and class diagrams describe its data; the use case, activity, sequence and statechart diagrams describe its behaviour; and the package, component and deployment diagrams describe its structure. All UML models were prepared in StarUML.

3.6.1 Event Table

The event table lists every external event the system responds to, the trigger that raises it, the processing the system performs, and the response it returns.

Event	Trigger	Action Performed	System Response
Register	Visitor submits the registration form	Validates the input, rejects a duplicate email, hashes the password and stores the account	Account created and the user signed in
Login	User submits email and password	Verifies the credentials against the stored hash and signs an access token	Token issued; user dashboard displayed
Admin Login	Administrator submits credentials on the hidden route	Verifies the credentials and additionally checks the administrative role	Administrative dashboard displayed
Browse Events	User opens the events listing	Retrieves events whose status is published or sold out	List of events displayed
View Event	User selects an event	Retrieves the event by its slug together with its reviews	Event detail page displayed
Publish Event	User submits the event form	Validates the details, derives a unique slug and stores the event	Event created and listed
Book Ticket	User confirms a booking quantity	Checks availability, decrements the ticket count atomically, creates the booking and records the payment	Booking confirmed and reference issued
Make Payment	User completes the demo gateway or submits a UPI reference	Records the payment in the ledger and sets the booking state accordingly	Payment recorded; ticket activated or held for review
Confirm UPI Payment	Administrator approves a pending transfer	Marks the payment paid, records the confirming administrator and confirms the booking	Ticket activated
Cancel Booking	User or administrator cancels a booking	Marks the booking cancelled, refunds the ledger entry and restores the tickets to the event	Booking cancelled; inventory restored
View Bookings	User opens the bookings page	Retrieves the bookings belonging to that account	Booking history displayed
Download Ticket	User requests the ticket PDF	Encodes the booking as a QR code, rasterises the ticket and generates a PDF	Ticket downloaded
Submit Feedback	Attendee submits a rating and review	Verifies a confirmed booking exists and stores one review per user per event	Review stored and shown on the event
Manage Users	Administrator opens the user console	Retrieves all registered accounts without password hashes	User list displayed
Manage Bookings	Administrator creates, cancels or transfers a booking	Applies the operation and updates the affected event and ledger entries	Booking records updated
View Payments	Administrator opens the payment ledger	Retrieves every payment with its user, event and confirming administrator	Payment ledger displayed
Logout	User or administrator signs out	Clears the stored token and session state	Session ended; public view restored

3.6.2 Entity Relationship Diagram

The entity-relationship diagram records the entities the system stores and the relationships between them. A User places many Bookings and writes many Reviews; an Event receives many Bookings and many Reviews; each Booking settles through exactly one

Payment; and every Event is created by exactly one User. The cardinalities shown are what the application enforces, with the additional restriction that a User may hold at most one Review per Event.

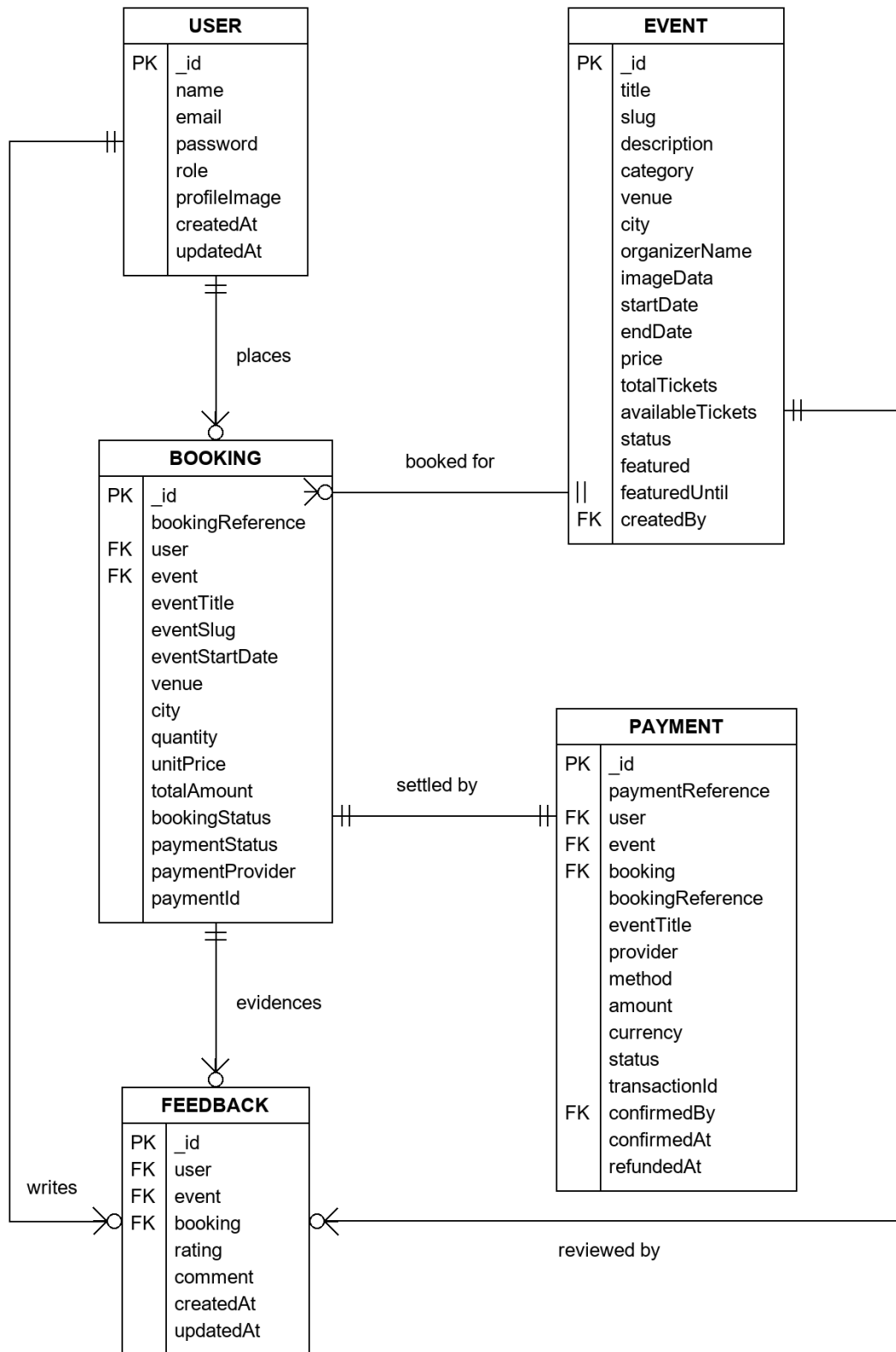


Fig. 3.6.2: Entity Relationship Diagram

3.6.3 Class Diagram

The class diagram presents the entity classes with their attributes and the operations defined on them. The User class carries the hashing hook, the password comparison method and the projection that strips the hash before a document is returned; the Event class carries the slug derivation and lifecycle status; the Booking class carries the denormalised event snapshot; and the Payment class carries the provider, method and status enumerations that describe a settlement attempt.

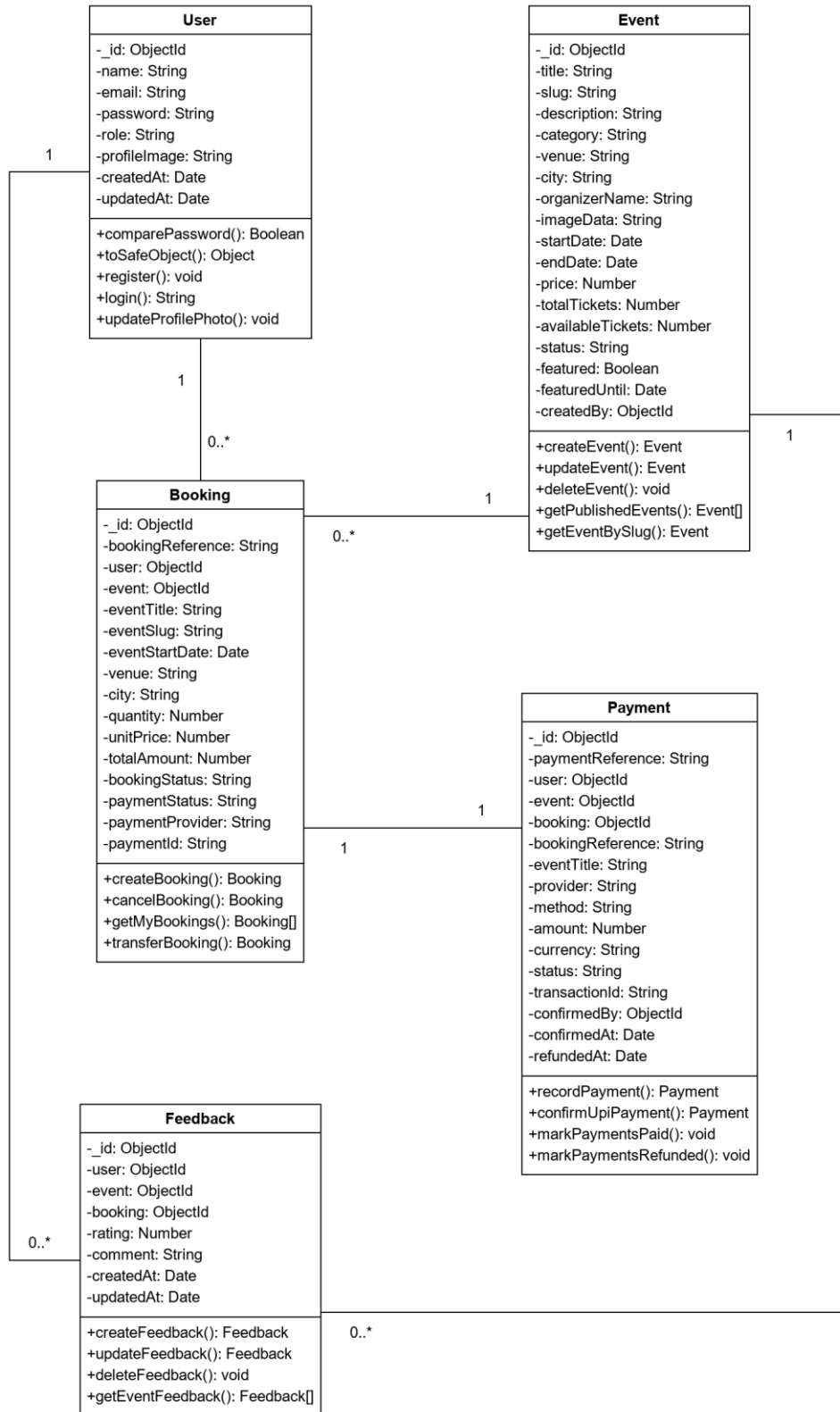


Fig. 3.6.3: Class Diagram

3.6.4 Object Diagram

The object diagram shows a representative snapshot of the system at run time — particular instances of the classes above, with concrete attribute values and the links between them. It illustrates how a single booking ties one user to one event and one payment record.

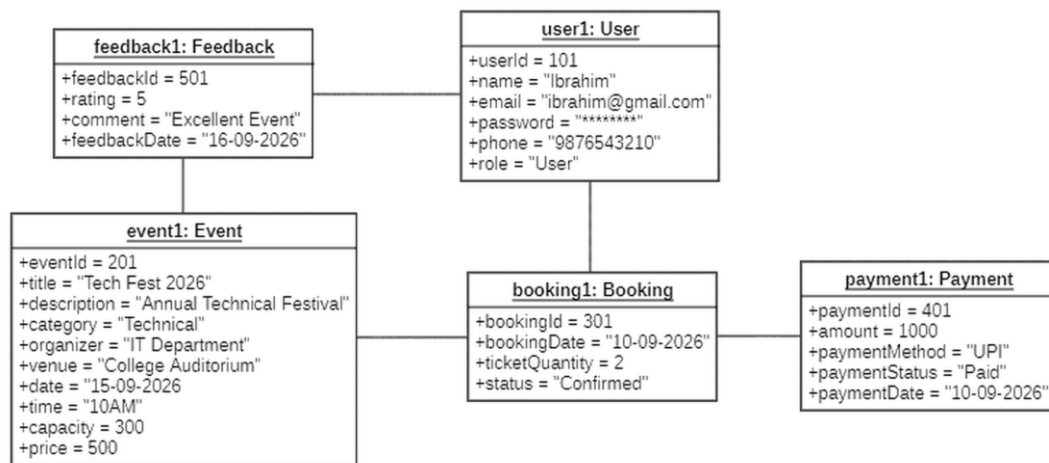


Fig. 3.6.4: Object Diagram

3.6.5 Use Case Diagram

The use case diagram identifies the two actors the system recognises and the cases each may invoke. The registered user registers, signs in, browses and searches events, publishes and manages their own events, books tickets, makes payment, views bookings, downloads tickets, cancels bookings and submits feedback. The administrator signs in separately and manages users, events, bookings, payments and reviews. The cases shared between the two actors — signing in, browsing events — are shown as such.

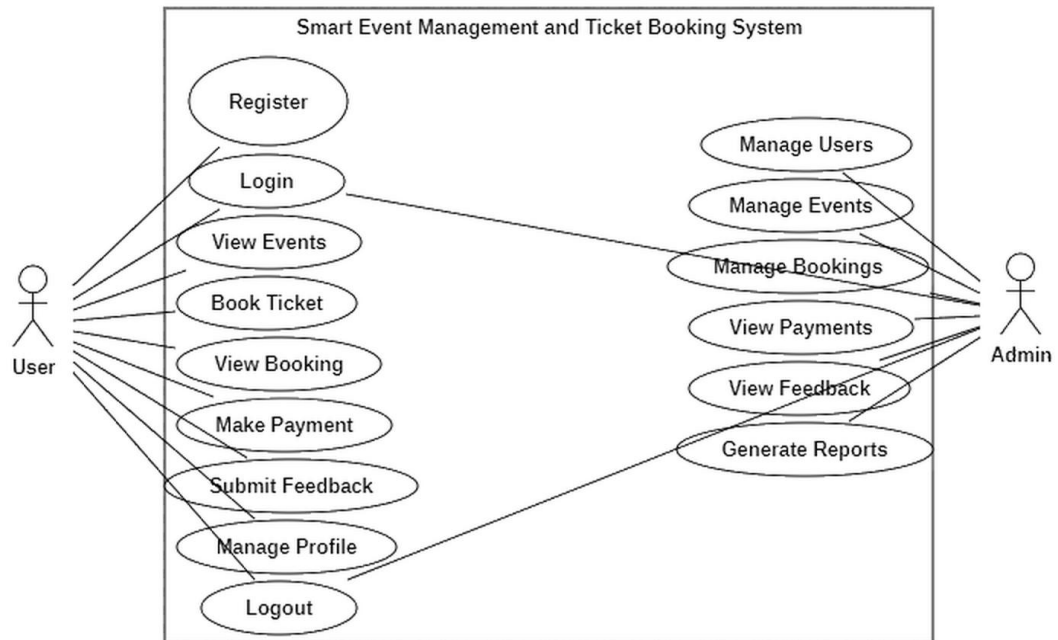


Fig. 3.6.5: Use Case Diagram

3.6.6 Activity Diagram

The activity diagram traces the flow of control through the principal workflow: a user signs in, browses events, selects one, enters a ticket quantity, and the system validates availability before creating the booking, decrementing inventory, recording the payment and issuing the ticket. The decision nodes show where the flow branches — on insufficient availability, on a free versus a priced event, and on the choice of payment channel.

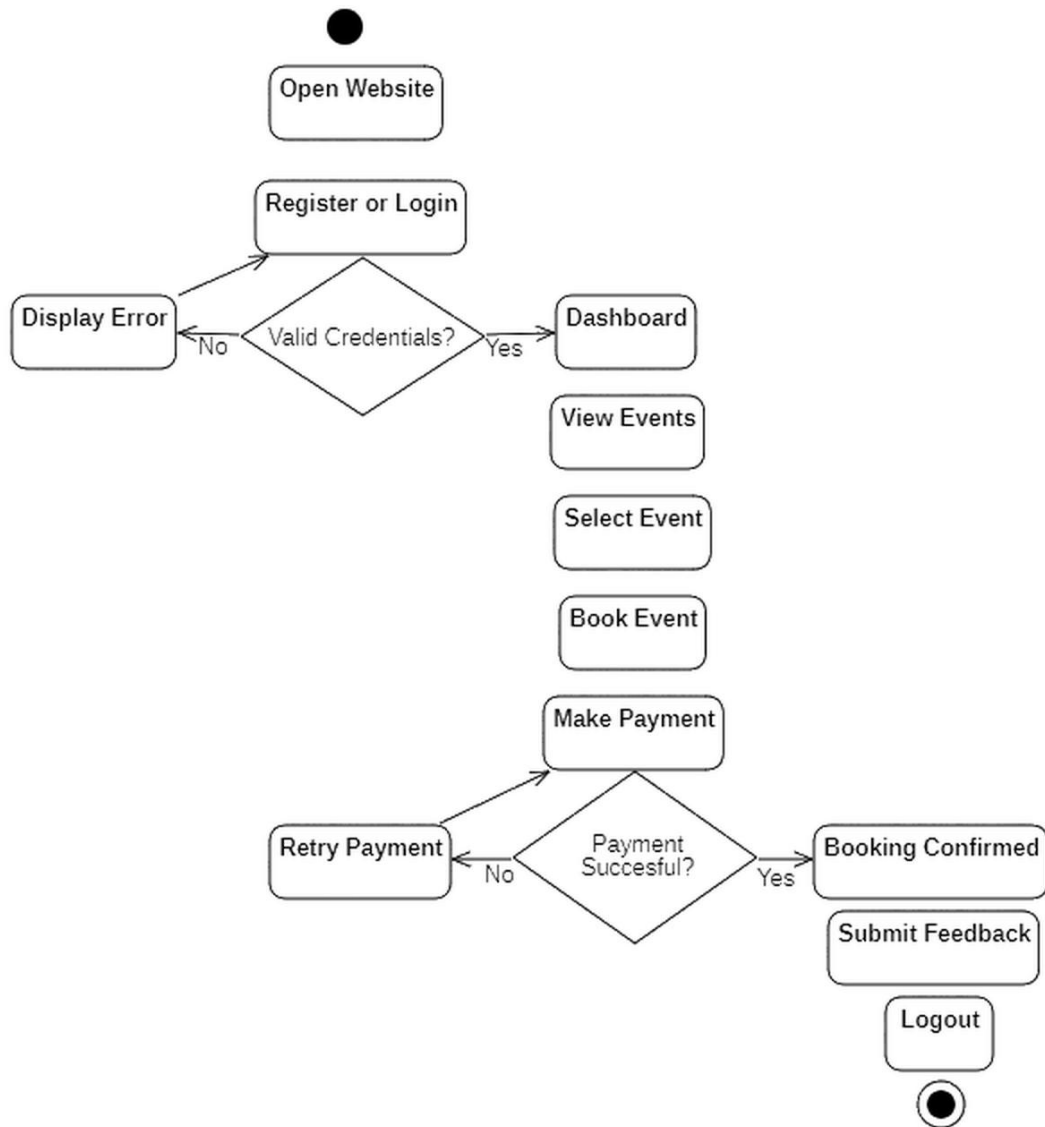


Fig. 3.6.6: Activity Diagram

3.6.7 Sequence Diagram

The sequence diagram shows the messages exchanged between the user, the React client, the Express API, the authentication middleware and the MongoDB database over the lifetime of a booking request. It makes the ordering explicit: the token is verified and the role checked before the controller runs, and the inventory decrement and the payment record are written before the response is returned.

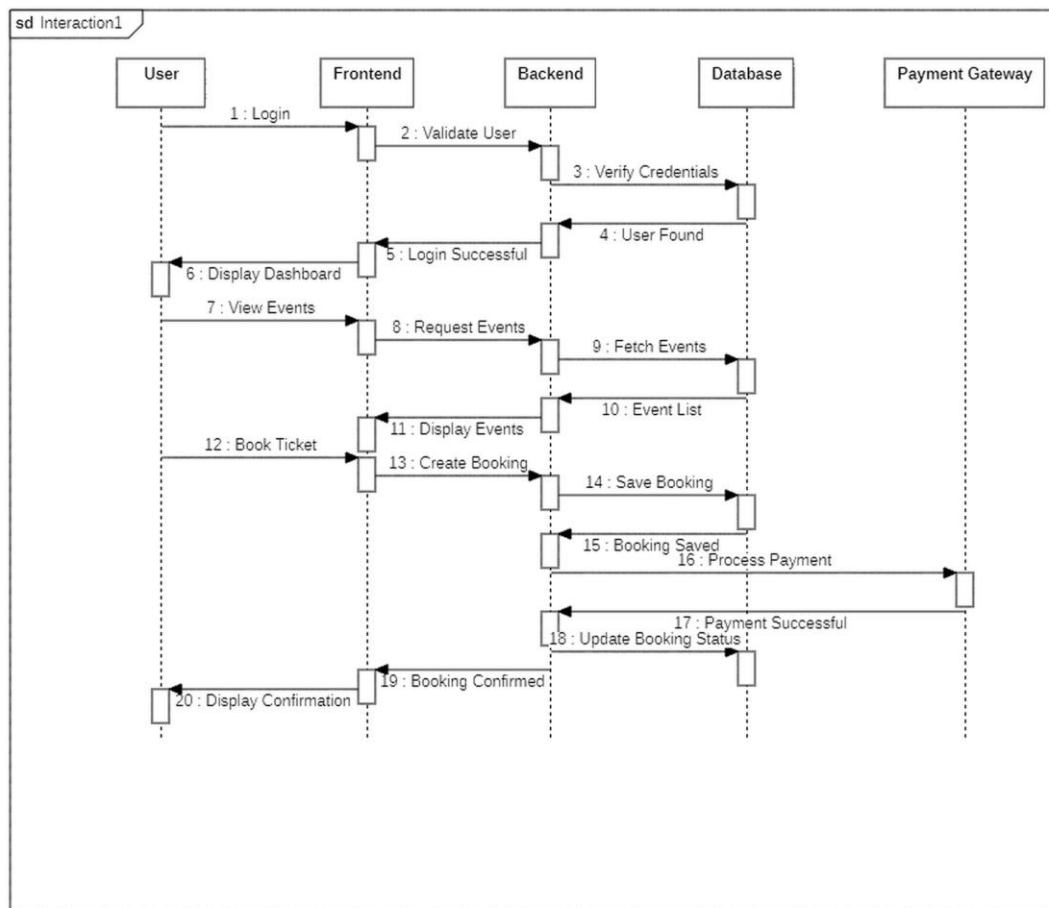


Fig. 3.6.7: Sequence Diagram

3.6.8 Statechart Diagram

The statechart diagram models the lifecycle of a booking and of the event it is made against. A booking moves from pending payment to confirmed on settlement, and from either state to cancelled; an event moves from draft to published, from published to sold out when inventory is exhausted, and back to published when a cancellation returns stock. The transitions that must be reversible — and were the source of the defects discussed in Section 6.1.2 — are visible here as paired arrows.

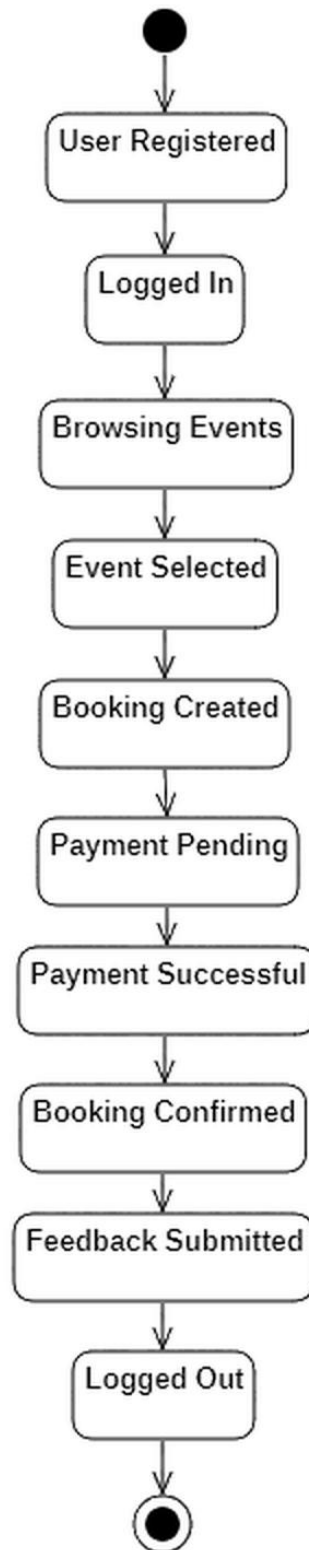


Fig. 3.6.8: Statechart Diagram

3.6.9 Package Diagram

The package diagram shows the organisation of the codebase into packages and the dependencies between them. On the client, pages depend on components, services and context; on the server, routes depend on middleware and controllers, and controllers depend on models and utilities. The dependency arrows run in one direction only, which is what keeps a change in one layer from propagating outwards.

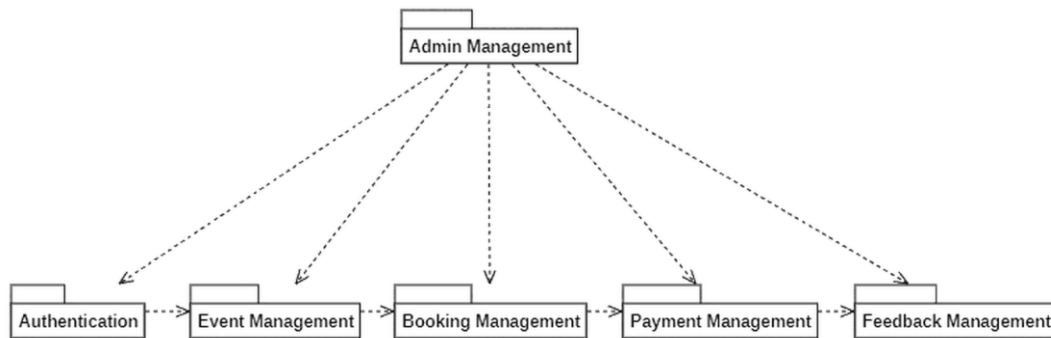


Fig. 3.6.9: Package Diagram

3.6.10 Deployment Diagram

The deployment diagram shows the physical arrangement of the deployed system: the browser executing the React bundle, the Vercel edge network serving that bundle, the Railway node running the Express process, and the MongoDB Atlas cluster holding the data. The communication paths between the nodes are HTTPS.

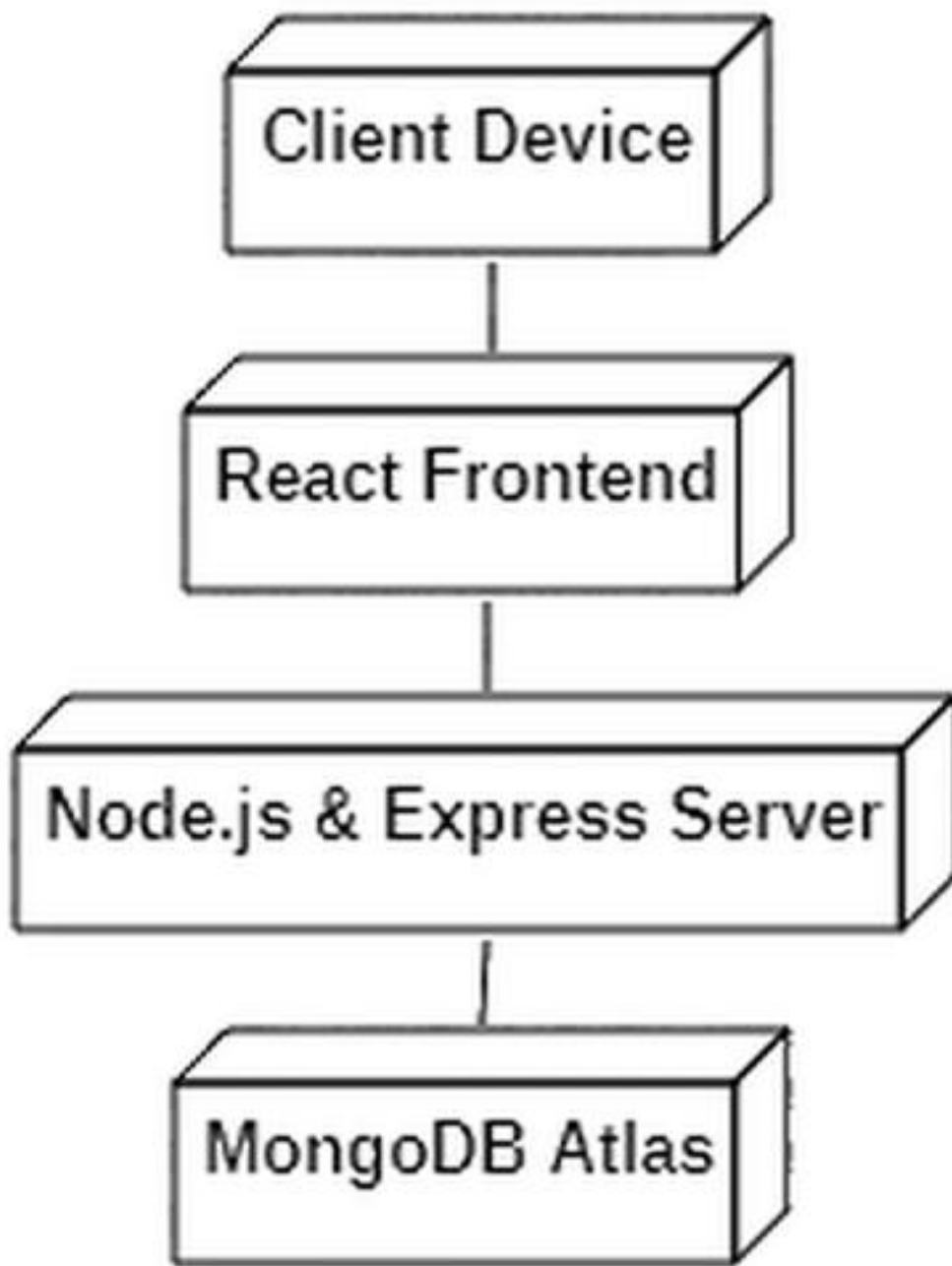


Fig. 3.6.10: Deployment Diagram

3.6.11 Data Flow Diagram

The data flow diagram shows how data moves between the external entities, the processes that transform it and the data stores that hold it. The context level places the system as a single process between the user and the administrator; the level-1 decomposition shown here expands it into the authentication, event management, booking, payment, ticketing,

feedback and administration processes, each reading from and writing to the corresponding collection.

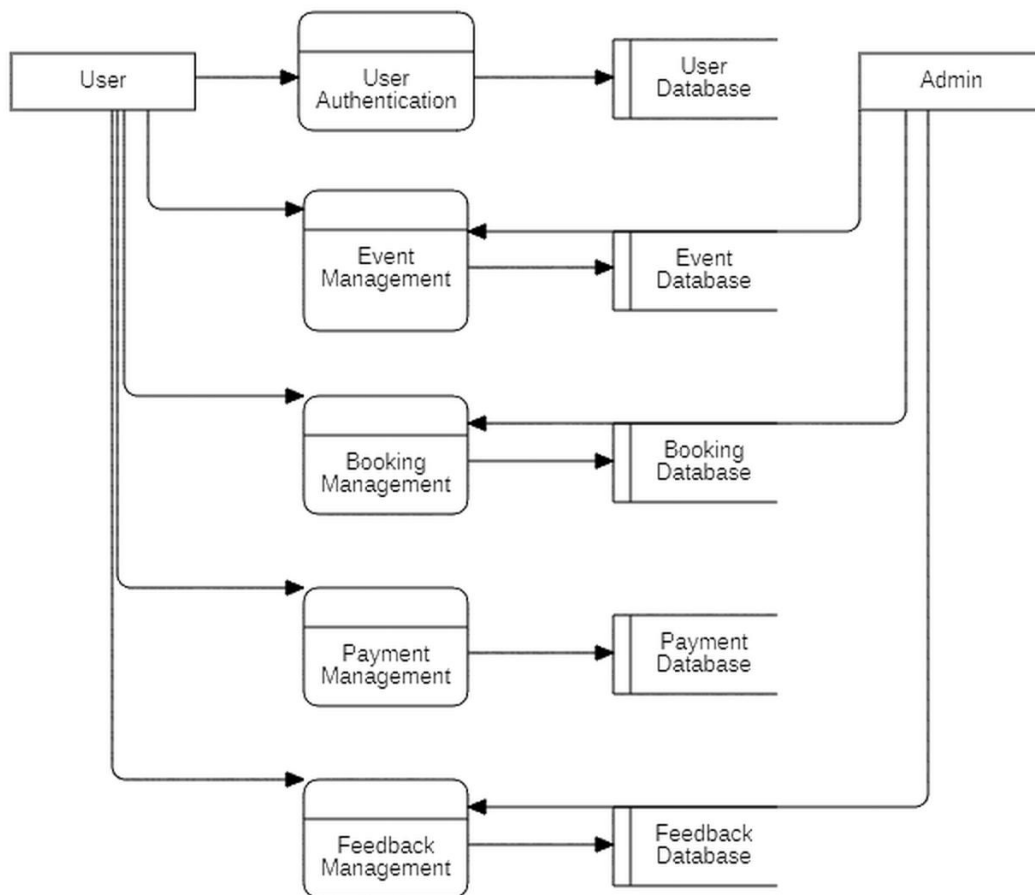


Fig. 3.6.11: Data Flow Diagram

SYSTEM DESIGN

4.1 Basic Modules

The system is decomposed into seven functional modules. Each owns a distinct area of responsibility, exposes a defined set of operations, and communicates with the others only through those operations. The decomposition follows the package structure presented in Section 3.6.9 and is mirrored directly in the codebase, where each module corresponds to a controller, a set of routes and — where it owns persistent data — a Mongoose model.

4.1.1 Module 1: User Management and Authentication

Purpose. To establish and verify identity, and to enforce the distinction between an ordinary user and an administrator.

Operations. Account registration; user sign-in; administrator sign-in; retrieval of the currently authenticated account; profile image update; sign-out; and administrative retrieval of the registered user list.

Processing. Registration validates the submitted name, email and password, rejects an email that already exists, and stores the account with the password hashed by bcrypt. Sign-in retrieves the account by email, compares the supplied password against the stored hash using bcrypt's constant-time comparison, and on success signs a JSON Web Token carrying the account identifier and role. Every subsequent protected request presents that token, which middleware verifies before the request reaches a controller.

Data. The users collection.

Validation. Name of at least two characters; a syntactically valid email address, stored lowercase and unique across the collection; a password of at least six characters matching its confirmation; role constrained to the values user or admin.

4.1.2 Module 2: Event Management

Purpose. To allow events to be published, maintained and discovered.

Operations. Create event; update event; delete event; list published events; list featured events; retrieve an event by slug; list the events belonging to the signed-in account; list all events for administrators; set and withdraw featured status.

Processing. Creation validates the submitted details, derives a URL-friendly slug from the title, guarantees the slug's uniqueness by testing for collision and appending a numeric suffix, initialises the available ticket count to the total, and stores the event with a reference to the account that created it. Update and delete first verify that the caller either owns the event or holds the administrator role. The public listing returns only events whose status is published or sold out; featured placement is additionally bounded by an expiry timestamp, so a featured event withdraws itself once its window has passed.

Data. The events collection.

Validation. Title present and at most 120 characters; description present and at most 1200 characters; category, venue, city and organiser name present; start and end dates present, well-formed and correctly ordered; price zero or greater; total tickets at least one; available tickets not exceeding total; status drawn from the permitted set; featured duration between one and twenty-four hours.

4.1.3 Module 3: Booking Management

Purpose. To reserve tickets against an event and to keep ticket inventory consistent with the set of active bookings.

Operations. Create booking; list the signed-in user's bookings; retrieve a single booking; cancel a booking; and, for administrators, list all bookings and create, cancel or transfer a booking on a user's behalf.

Processing. A booking request is satisfied by a single conditional update that matches the event only if it is published and holds at least the requested quantity, and decrements the available count in the same operation. Because the check and the decrement are one indivisible database operation, two simultaneous requests for the last remaining ticket cannot both succeed. If the update matches, the booking is created with a generated reference code and a snapshot of the event's title, slug, start date, venue and city, and the event's status becomes sold out if the count has reached zero. Cancellation reverses this: the booking's status becomes cancelled, its payment is marked refunded, the quantity is returned to the event, and an event previously marked sold out returns to published.

Data. The bookings collection; reads from events and users; writes to payments through Module 4.

Validation. The caller must be authenticated; the event must exist and be bookable; quantity must be a positive integer not exceeding availability; a priced event must be settled through the payment module rather than booked directly; a booking already cancelled cannot be cancelled again; a cancelled booking cannot be transferred.

4.1.4 Module 4: Payment Management

Purpose. To settle the amount owed on a booking and to maintain an auditable record of every payment attempt.

Operations. Complete a demo-gateway payment; submit a manual UPI transfer reference; confirm a reviewed UPI payment (administrator); list the payment ledger (administrator).

Processing. Three channels are supported. A zero-priced event produces a payment record with status `not_required` and the booking is confirmed immediately. The demo gateway records a payment with status `paid` and confirms the booking in the same operation. The manual UPI channel records a payment with status `pending` against the transfer reference the customer supplies, leaving the booking in `pending_payment` until an administrator reviews the transfer and confirms it, at which point the payment becomes `paid`, the confirming administrator and timestamp are recorded, and the booking is confirmed. All three channels write through one shared ledger helper, so the mapping from provider to method cannot drift between them.

Data. The payments collection; updates bookings.

Validation. The referenced event must exist, be published and be priced; the quantity must be a positive integer within availability; a UPI transfer reference must be between six and one hundred characters; a payment not in the pending state cannot be confirmed.

4.1.5 Module 5: Ticketing

Purpose. To issue the artefact the attendee actually carries.

Operations. Render a ticket for a booking; encode its identifying details as a QR code; export the ticket as a PDF.

Processing. The ticket component renders the booking reference, event title, attendee name, date, venue, city and quantity, alongside a QR code encoding those same details. When the holder requests a copy, the rendered element is rasterised to a canvas and

placed into a PDF sized to fit it. All three supporting libraries are loaded on demand, so the code is fetched only by users who actually request a ticket.

Data. Reads from bookings; writes nothing.

Validation. The ticket displays the booking's current state, so a cancelled or payment-pending booking is visibly marked as such rather than presented as a valid entry pass.

4.1.6 Module 6: Feedback Management

Purpose. To let an attendee rate and review an event after attending, and to let administrators moderate what has been submitted.

Operations. Submit a review; edit one's own review; withdraw one's own review; list the reviews for an event with their aggregate rating; retrieve the signed-in user's own review and eligibility for a given event; list all reviews and remove any review (administrator).

Processing. Submission first confirms that the calling user holds a confirmed booking for the event in question, which is what ties a review to genuine attendance rather than to mere registration. A compound unique index on the user and event fields enforces at most one review per user per event at the database level, so a duplicate cannot be created even by two simultaneous requests. The event page displays the aggregate rating alongside the individual reviews.

Data. The feedback collection; reads from bookings and events.

Validation. The caller must hold a confirmed booking for the event; the rating must be a whole number between one and five; the comment is required and may not exceed 800 characters; at most one review per user per event.

4.1.7 Module 7: Administration

Purpose. To give administrators oversight of the platform.

Operations. List all events, bookings, payments, reviews and registered users; edit or delete any event; control featured placement; confirm reviewed UPI payments; create, cancel or transfer bookings on a user's behalf; remove a review; and query system health.

Processing. Every administrative route passes through the token verification middleware and then through role authorisation, so administrative capability is enforced at the API layer rather than assumed from the interface. The administrator account itself is

provisioned from environment configuration when the server first connects to the database, which avoids shipping any credential in the codebase. The administrative sign-in page is deliberately unlinked from the public navigation and is reached through a hidden trigger.

Data. Reads across all collections; writes to events, bookings, payments and feedback.

Validation. The caller's role must be admin on every operation in this module.

4.2 Data Design

4.2.1 Schema Design

The system stores its data in MongoDB, a document-oriented database, accessed through the Mongoose object data modelling library. MongoDB itself does not enforce a structure on the documents within a collection; Mongoose adds a schema layer above it that declares the permitted fields, their types, whether they are required and what values they may hold. The application therefore has the flexibility of a document store while retaining the field-level guarantees usually associated with a relational system.

The database is organised into five collections — users, events, bookings, payments and feedback — each backed by a schema defined under `backend/models/`. Every schema is declared with the `timestamps` option, so each document automatically carries `createdAt` and `updatedAt` fields maintained by Mongoose rather than by application code.

Collection	References	Purpose
users	—	Registered user and administrator accounts
events	users (createdBy)	Event listings published on the platform
bookings	users, events	Ticket reservations made against an event
payments	users, events, bookings	Settlement attempts recorded against a booking
feedback	users, events, bookings	Ratings and reviews submitted by attendees

4.2.1.1 User Schema

The User schema stores authentication credentials and the role that separates an ordinary user from an administrator.

Field	Type	Constraints	Description
<code>_id</code>	ObjectId	primary key, generated	Unique document identifier

Field	Type	Constraints	Description
name	String	required, 2–50 characters	Display name of the account holder
email	String	required, lowercase, validated, unique, pattern-	Login identifier
password	String	required, minimum 6 characters	bcrypt hash; never stored in plain text
role	String (enum)	user admin, default user	Determines access to administrative routes
profileImage	String	optional, default empty	Base64-encoded avatar, limited to 1 MB
createdAt, updatedAt	Date	auto-managed	Creation and last-modified timestamps

A pre('save') hook hashes the password with bcrypt using ten salt rounds whenever it is created or modified, so a plain-text password never reaches the database. A comparePassword() instance method performs the corresponding comparison at login, and a toSafeObject() method strips the hash before any user document is returned to a client.

4.2.1.2 Event Schema

Field	Type	Constraints	Description
title	String	required, max 120 characters	Event name
slug	String	required, unique	URL-friendly identifier derived from the title
description	String	required, max 1200 characters	Full event description
category	String	required, max 60 characters	Event category
venue, city	String	required, max 120 / 60 characters	Physical location
organizerName	String	required, max 80 characters	Organising entity
imageData	String	optional, PNG/JPG/WEBP	Base64-encoded banner image
startDate, endDate	Date	required, end not before start	Start and end date-time
price	Number	required, minimum 0	Ticket price
totalTickets	Number	required, minimum 1	Capacity at listing time
availableTickets	Number	required, minimum 0	Remaining unsold tickets
status	String (enum)	draft published sold_out cancelled	Current lifecycle state
featured, featuredUntil	Boolean, Date	optional, window of 1–24 hours	Homepage featured placement and its expiry
createdBy	ObjectId → User	optional	Account that published the event

4.2.1.3 Booking Schema

Field	Type	Constraints	Description
bookingReference	String	required, unique	Human-readable code, e.g. BKG-20260806-D5GIU2
user	ObjectId → User	required	Account that made the booking
event	ObjectId → Event	required	Event being booked
eventTitle, eventSlug, eventStartDate, venue, city	String, Date	required, denormalised	Snapshot of the event at booking time
quantity	Number	required, minimum 1	Number of tickets
unitPrice, totalAmount	Number	required, minimum 0	Price per ticket and computed total
bookingStatus	String (enum)	pending_payment confirmed cancelled	Lifecycle state of the booking
paymentStatus	String (enum)	pending paid refunded not_required	Payment state
paymentProvider	String (enum)	demo upi_manual free manual	Channel used
paymentId, paymentOrderId	String	optional	External payment identifiers

4.2.1.4 Payment Schema

Field	Type	Constraints	Description
paymentReference	String	required, unique	Human-readable code, e.g. PAY-20260825-YZ0FKL
user, event, booking	ObjectId	required	References tying the payment to its context
bookingReference, eventTitle	String	required, denormalised	Snapshot fields for display without a join
provider	String (enum)	demo upi_manual free manual	Payment provider
method	String (enum)	demo_gateway upi_transfer offline none	Payment method
amount	Number	required, minimum 0	Transaction amount
currency	String	default INR, uppercase	ISO currency code
status	String (enum)	pending paid refunded failed not_required	Current transaction state
transactionId	String	optional	Provider-side reference
confirmedBy, confirmedAt	ObjectId → User, Date	optional	Administrator who confirmed the payment, and when
refundedAt	Date	optional	Timestamp of refund, if applicable

The Payment schema additionally declares indexes on {booking, createdAt}, {user, createdAt} and {status}, so that the three queries the application actually issues — a booking’s payment record, a user’s payment history, and filtering by status in the administrative ledger — remain efficient as the collection grows.

4.2.1.5 Feedback Schema

Field	Type	Constraints	Description
user	ObjectId → User	required	Account that wrote the review
event	ObjectId → Event	required	Event being reviewed
booking	ObjectId → Booking	required	The confirmed booking that establishes attendance
rating	Number	required, 1–5	Star rating awarded
comment	String	required, max 800 characters	Written review
createdAt, updatedAt	Date	auto-managed	Submission and last-edited timestamps

A compound unique index on {user, event} enforces the one-review-per-user-per-event rule at the database level, so a duplicate cannot be created even if two submissions arrive simultaneously and both pass the application’s own check.

4.2.1.6 Design Rationale

Denormalisation in preference to joins. MongoDB does not perform relational joins, and Mongoose’s populate() — its nearest equivalent — issues a separate query for each reference resolved. To avoid that cost on frequently-read collections, Booking and Payment documents duplicate a small set of Event fields at the moment they are created. This buys more than performance. Because the copy is taken at purchase time, a historical booking continues to show what the customer actually bought even after the event listing is edited. Without it, changing an event’s venue would silently rewrite the record of every past booking against that event.

A single collection for both roles. Ordinary users and administrators are stored in the same users collection and distinguished only by the role field, rather than being separated into two collections. Access control is therefore enforced consistently at one place — the route middleware — instead of being implied by which collection a record happens to live in.

Enumerated status fields. Every lifecycle-sensitive field — `Event.status`, `Booking.bookingStatus` and `paymentStatus`, and `Payment.status` — is constrained by a Mongoose enumeration validator. This pushes state-machine validation into the schema itself, so an invalid value is rejected at the point of writing rather than discovered later by whatever code reads it.

Referential integrity without foreign keys. MongoDB enforces no foreign-key constraints. Referential correctness is maintained instead at the application layer: controllers create a `Booking`, a `Payment` or a `Feedback` record only after confirming that the referenced documents exist, and every reference field is marked required so that a missing reference cannot be persisted through the normal application flow.

4.2.2 Data Integrity and Constraints

Data integrity in this system is enforced at four levels, each catching a different class of error.

Schema-level constraints are declared on the Mongoose models and applied on every write. Required fields cannot be omitted. The email field carries a uniqueness constraint, which prevents duplicate accounts even if two registration requests arrive simultaneously and both pass the application's own duplicate check, and the feedback collection carries a compound uniqueness constraint that does the same for reviews. String fields carry length bounds, numeric fields carry minimum values — price at least zero, `totalTickets` at least one, `availableTickets` at least zero, rating between one and five — and every status field is restricted to an enumerated set. The email field is additionally validated against a pattern and normalised to lowercase before storage, so that address casing cannot produce two accounts that are the same address in practice.

Application-level constraints are enforced in the controllers, and cover the rules a schema cannot express. Registration rejects an email already present in the collection with a specific error rather than allowing the database constraint to surface as an opaque failure. Booking creation rejects a quantity exceeding the event's available tickets, and rejects a direct booking against a priced event that has not been paid for. Cancellation rejects a booking already cancelled. Payment confirmation rejects a payment not in the pending state. Feedback submission rejects a review from a user holding no confirmed booking for that event. Update and delete operations on an event verify that the caller either created that event or holds the administrator role.

Invariant maintenance concerns the one relationship the system must never allow to drift: the correspondence between an event's available ticket count and the set of active bookings against it. Booking creation is expressed as a single conditional update — the event is matched only if it is published and holds at least the requested quantity, and the decrement is applied in the same operation — so the check and the decrement cannot be separated by a competing request. Cancellation restores the count and reverses the sold-out status if it was set. The count and the bookings therefore cannot diverge.

Credential integrity is maintained by construction. The password field is hashed by a pre-save hook that runs on every create and every modification, so there is no code path through which a plain-text password can be written — not even one that bypasses the controller. Verification uses bcrypt's own comparison rather than string equality, and the `toSafeObject()` projection ensures the hash is never included in a response.

4.3 User Interface Design

4.3.1 Design Principles

Four principles governed the interface design.

Progressive disclosure of capability. A visitor who has not signed in sees the landing page, the events listing and individual event pages. Booking, ticket download, event publishing, review submission and administrative consoles appear only once the corresponding role is established. Capability is never merely hidden — the same distinction is enforced at the API — but the interface avoids presenting controls that would fail.

Consistency of structure. Every authenticated page shares a common layout: a persistent navigation bar carrying the brand, the primary links, the theme toggle and the account menu; a content region; and a footer. Two layout components, `MainLayout` and `AuthLayout`, implement this, so the structure cannot drift between pages.

Responsiveness by default. The interface is built on Bootstrap's twelve-column grid with mobile-first breakpoints. Event listings collapse from a multi-column grid to a single column on narrow viewports; the navigation collapses to a toggle; tables in the administrative consoles scroll horizontally within their container rather than forcing the page to scroll.

Immediate, specific feedback. Every form validates on the client before submission and displays server-side validation messages inline. Actions that take time show a loading state rather than appearing unresponsive, and every destructive action — cancelling a booking, deleting an event, withdrawing a review — requires confirmation.

4.3.2 Input Design

Form	Fields	Validation applied
Registration	Name, email, password, confirm password	All required; name ≥ 2 characters; email pattern-checked and checked for duplication; password ≥ 6 characters and must match its confirmation
User sign-in	Email, password	Both required; email pattern-checked; credentials verified against the stored hash
Administrator sign-in	Email, password	As above, with an additional role check
Create / edit event	Title, description, category, organiser, venue, city, banner image, start and end date-time, price, total tickets	All required except the banner; length bounds on text fields; end date not before start; price ≥ 0 ; tickets ≥ 1 ; image restricted by type and size
Booking	Ticket quantity	Positive whole number; must not exceed the tickets currently available
Demo gateway payment	Confirmation of the displayed amount	Amount must match the booking total; the event must be published and priced
Manual UPI payment	Transfer reference	Required; 6–100 characters; recorded against the booking for administrative review
Feedback	Star rating, written review	Rating a whole number 1–5; comment required and at most 800 characters; the user must hold a confirmed booking
Profile image	Image file	PNG, JPEG or WebP; encoded and stored against the account; maximum 1 MB

4.3.3 Output Design

The system's outputs are the landing page with its featured selection; the event listing and detail pages; the user dashboard; the booking history; the QR-coded ticket and its PDF export; payment confirmations; the aggregate rating and review list on each event; the administrative consoles for events, bookings, payments, feedback and users; and the validation and error messages produced when an operation cannot be completed. Every output is rendered through the same Bootstrap-based component set, so presentation is consistent across desktop, tablet and mobile, and across the light and dark themes.

4.3.4 Screen Design

Screen	Route	Purpose
--------	-------	---------

Screen	Route	Purpose
Home	/	Landing page with the featured event selection
Events	/events	Listing of all published events
Event Details	/events/:slug	Full event information, booking controls, reviews and ratings
Sign in / Sign up	/login, /register	User authentication
Administrator sign-in	/admin/login	Unlinked administrative authentication
User Dashboard	/dashboard	Account summary and navigation
My Events	/my-events	Events published by the signed-in user
Bookings	/bookings	Booking history and QR-coded tickets
Admin Dashboard	/admin/dashboard	Event oversight and featured placement
Admin Bookings	/admin/bookings	Booking creation, cancellation and transfer
Admin Payments	/admin/payments	Payment ledger and UPI confirmation
Admin Feedback	/admin/feedback	Review moderation
Not Found	*	Fallback for an unmatched route

4.4 Security Issues

Security in this system is addressed at six layers. Each is described below together with the specific threat it is intended to counter.

4.4.1 Password Storage

Passwords are never stored or transmitted in recoverable form. A pre('save') hook on the User schema hashes the password with bcrypt using ten salt rounds before the document is written, and because the hook is attached to the schema rather than invoked by a controller, there is no code path that can write a plain-text password. bcrypt incorporates a distinct random salt into every hash, so two accounts sharing a password produce different stored values and precomputed lookup tables are useless. Verification uses bcrypt's constant-time comparison rather than string equality, which prevents an attacker from inferring a password character by character from response timing.

This counters the compromise of the database itself: an attacker holding the users collection obtains hashes whose reversal costs the bcrypt work factor per candidate password tested.

4.4.2 Authentication

Authentication is stateless and token-based. A successful sign-in produces a JSON Web Token signed with a secret held in environment configuration and carrying the account identifier and role. The client presents that token in the Authorization header of every subsequent request; the protect middleware verifies the signature, loads the corresponding account with the password field explicitly excluded, and attaches it to the request. A request with no token, a malformed token, an expired token or a token whose subject no longer exists is rejected with 401 before any controller runs.

4.4.3 Authorisation

Authentication establishes who is calling; authorisation determines what they may do. The `authorize()` middleware is a factory returning a guard for a given set of roles, and it is composed into the route definition after `protect`. Administrative routes are therefore declared as the path followed by `protect`, then `authorize('admin')`, then the controller.

Two aspects of this deserve emphasis. First, authorisation is enforced at the API and not in the interface: hiding an administrative link does not protect the endpoint behind it, and the endpoint is protected independently. Second, ownership checks supplement role checks where roles alone are insufficient — updating or deleting an event verifies that the caller created it or is an administrator, and editing a review verifies that the caller wrote it, so one user cannot modify another's records merely by holding a valid token.

4.4.4 Configuration and Secret Management

The database connection string, the token signing secret and the administrator bootstrap credentials are supplied through environment variables loaded at start-up, and the environment file is excluded from version control. No credential appears in the codebase. The administrator account is created from that configuration when the server first connects to the database, which avoids the alternative of shipping a seeded account with a known password.

4.4.5 Transport and Origin Control

Both deployed tiers are served over HTTPS, so credentials and tokens are encrypted in transit. Cross-Origin Resource Sharing is configured to accept browser requests only from

the configured frontend origin, which prevents a page hosted elsewhere from issuing authenticated requests against the API using a victim's stored token.

4.4.6 Input Handling and Error Disclosure

Every endpoint validates its input before acting on it, and Mongoose's schema validators provide a second layer beneath that. Uploaded images are checked against a permitted MIME prefix and a maximum size before being stored. Because MongoDB is queried through Mongoose's typed API rather than by string concatenation, the injection route familiar from SQL systems does not arise in the same form. A centralised error middleware formats every failure into a consistent JSON response, so stack traces and internal paths are never returned to a client in production, and a not-found middleware handles unmatched routes uniformly.

4.4.7 Known Limitations

Two limitations are worth stating explicitly. Because authentication is stateless, a token cannot be revoked before it expires without reintroducing server-side state; a compromised token remains valid for the remainder of its lifetime. And rate limiting is not currently applied to the authentication endpoints, so the system does not itself slow an automated credential-guessing attempt — although bcrypt's cost factor makes such an attempt expensive regardless. Both are noted in Section 7.2 and addressed in Section 7.3.

4.5 Test Cases Design

Test cases were designed to cover each module's normal path, its boundary conditions and its failure modes. The table below records the case identifier, the module under test, the scenario, the input supplied and the expected result. The outcomes obtained when these cases were executed are reported in Section 6.1.

ID	Module	Scenario	Input	Expected Result
TC-01	User Management	Register with valid details	Name, unused email, matching passwords	Account created and user signed in
TC-02	User Management	Register with a duplicate email	Email already registered	Rejected with a duplicate-account message
TC-03	User Management	Register with a malformed email	user@@mail	Rejected with a format message
TC-04	User Management	Register with mismatched passwords	Password and confirmation differ	Rejected with a mismatch message
TC-05	User Management	Register with a short password	4-character password	Rejected with a minimum-length message

ID	Module	Scenario	Input	Expected Result
TC-06	User Management	Sign in with valid credentials	Registered email and password	Token issued and dashboard displayed
TC-07	User Management	Sign in with a wrong password	Valid email, wrong password	Rejected with an invalid-credentials message
TC-08	User Management	Administrator sign-in	Bootstrap admin credentials	Administrative dashboard displayed
TC-09	User Management	Ordinary account on the admin sign-in page	Valid user credentials	Rejected with 403 Forbidden
TC-10	User Management	Access a protected route without a token	No Authorization header	401 Unauthorised
TC-11	User Management	Access an admin route as an ordinary user	Valid user token	403 Forbidden
TC-12	Event Management	Create an event with valid details	Complete event form	Event created with a unique slug
TC-13	Event Management	Create an event with zero tickets	totalTickets = 0	Rejected by the minimum-value validator
TC-14	Event Management	Create an event with a negative price	price = -100	Rejected by the minimum-value validator
TC-15	Event Management	Create an event ending before it starts	endDate earlier than startDate	Rejected with a date-order message
TC-16	Event Management	Create two events with the same title	Duplicate title	Both created with distinct slugs
TC-17	Event Management	Update an event owned by another user	Valid token, another user's event	403 Forbidden
TC-18	Event Management	Delete an own event	Owner's token	Event removed from the listing
TC-19	Event Management	Retrieve an event by slug	Valid published slug	Event detail returned
TC-20	Event Management	Retrieve an unpublished event publicly	Draft event slug	Not returned in the public listing
TC-21	Booking Management	Book within availability	Quantity 2, 10 available	Booking confirmed; availability becomes 8
TC-22	Booking Management	Book exactly the remaining tickets	Quantity 3, 3 available	Booking confirmed; event becomes sold out
TC-23	Booking Management	Book more than available	Quantity 5, 3 available	Rejected with an availability message
TC-24	Booking Management	Book zero tickets	Quantity 0	Rejected by quantity validation
TC-25	Booking Management	Book without signing in	No token	401 Unauthorised
TC-26	Booking Management	View own booking history	Valid user token	Only that user's bookings returned
TC-27	Booking Management	Cancel a booking	Confirmed booking	Cancelled; tickets restored; sold-out event republished
TC-28	Booking Management	Cancel an already-cancelled booking	Cancelled booking	Rejected with an already-cancelled message
TC-29	Payment Management	Book a free event	price = 0	Confirmed with payment status not_required
TC-30	Payment Management	Pay through the demo gateway	Valid paid booking	Payment recorded as paid; booking confirmed
TC-31	Payment Management	Submit a manual UPI reference	Transfer reference	Payment recorded as pending; booking held
TC-32	Payment Management	Submit a UPI reference that is too short	3-character reference	Rejected with a reference-length message
TC-33	Payment Management	Confirm a reviewed UPI payment	Administrator confirmation	Payment paid; booking confirmed; confirmer

ID	Module	Scenario	Input	Expected Result
				recorded
TC-34	Payment Management	Confirm the same payment twice	Already-confirmed payment	Rejected
TC-35	Payment Management	Verify the ledger after cancellation	Cancelled booking paid	Payment marked refunded
TC-36	Ticketing	Generate a ticket for a confirmed booking	Confirmed booking	Ticket rendered with a scannable QR code
TC-37	Ticketing	Export a ticket as PDF	Ticket view	PDF downloaded matching the on-screen ticket
TC-38	Feedback Management	Submit a review after attending	Rating 4 and a comment, confirmed booking held	Review stored and shown on the event
TC-39	Feedback Management	Submit a review without a booking	No confirmed booking for the event	Rejected as ineligible
TC-40	Feedback Management	Submit a second review for the same event	Review already exists	Rejected by the compound unique index
TC-41	Feedback Management	Submit an out-of-range rating	Rating 7	Rejected by the range validator
TC-42	Administration	List all events	Administrator token	Every event returned regardless of status
TC-43	Administration	List all bookings	Administrator token	Every booking on the platform returned
TC-44	Administration	List the payment ledger	Administrator token	Complete ledger returned
TC-45	Administration	List registered users	Administrator token	Every account returned without password hashes
TC-46	Administration	Transfer a booking	Source and target accounts	Booking reassigned to the target account
TC-47	Administration	Remove a review	Administrator token and review id	Review deleted and aggregate rating recalculated
TC-48	System	Query the health endpoint	GET /api/health	API and database status reported

IMPLEMENTATION AND TESTING

5.1 Implementation Approach

5.1.1 Strategy

Implementation followed an incremental, vertically-sliced approach. Rather than building the entire backend and then the entire frontend, each feature was carried from the database schema through the API to the interface before the next feature was started. Authentication was completed end to end first; then event publishing and discovery; then booking; then payments; then the administrative consoles; then ticketing; then feedback.

The reason for working this way is that it keeps the system runnable at every point. A horizontal approach — all models, then all controllers, then all pages — leaves nothing demonstrable until very late, and defers the discovery of interface-level problems until the API is already fixed. Working vertically meant that after the first week there was a working application that could be signed into, and every subsequent slice added capability to something that already ran. It also meant that API design decisions were validated against a real consumer immediately, rather than against an imagined one.

Within each slice the order was consistent: define or extend the Mongoose schema; write the controller and its validation; register the route with its middleware; test the endpoint in Postman across valid input, invalid input, missing token and wrong role; add the service function on the frontend; and finally build the interface that calls it. Testing the endpoint before building the interface is what makes a subsequent failure unambiguous — if the endpoint was verified in isolation, a failure in the browser is an interface problem.

5.1.2 Architecture

The implementation is a three-tier client–server application.

The presentation tier is a React single-page application. It is organised into pages/ for route-level views, components/ for reusable interface elements, layouts/ for the two shared page structures, context/ for global state, hooks/ for the consumers of that state, services/ for the API client and its per-domain call wrappers, and utils/ for helpers. This

separation means an interface change touches pages/ or components/, and an API change touches services/, without the two interfering.

The application tier is an Express server organised into routes/ declaring the endpoint table, controllers/ holding the request handling and business logic, models/ holding the Mongoose schemas, middleware/ holding the cross-cutting concerns, config/ holding the database connection and the administrator bootstrap, and utils/ holding token generation and the payment ledger helpers.

The data tier is a MongoDB Atlas cluster reached only by the application tier.

The tiers exchange JSON over HTTPS. Because the API holds no session state, identity travels in a signed token attached to each request.

5.1.3 API Endpoint Table

Method	Endpoint	Access	Purpose
POST	/api/auth/register	Public	Create a user account
POST	/api/auth/login	Public	Authenticate a user
POST	/api/auth/admin/login	Public	Authenticate an administrator
GET	/api/auth/me	User	Return the authenticated account
PUT	/api/auth/profile-photo	User	Update the profile image
GET	/api/auth/admin/users	Admin	List registered users
GET	/api/events	Public	List published and sold-out events
GET	/api/events/:slug	Public	Retrieve one event by slug
POST	/api/events	User	Publish an event
PUT	/api/events/:id	Owner / Admin	Update an event
DELETE	/api/events/:id	Owner / Admin	Delete an event
GET	/api/events/manage/list	User / Admin	List the caller's own events
GET	/api/events/admin/list	Admin	List every event
PATCH	/api/events/:id/featured	Admin	Withdraw featured placement
GET	/api/bookings	User	List the caller's bookings
POST	/api/bookings	User	Create a free-event booking
GET	/api/bookings/:id	User	Retrieve one booking
POST	/api/bookings/:id/cancel	User	Cancel a booking
GET	/api/bookings/admin/list	Admin	List every booking
POST	/api/bookings/admin/create	Admin	Book on a user's behalf
POST	/api/bookings/admin/:id/cancel	Admin	Cancel any booking
POST	/api/bookings/admin/:id/transfer	Admin	Transfer a booking
POST	/api/payments/demo	User	Settle through the demo gateway
POST	/api/payments/upi	User	Submit a UPI transfer reference
GET	/api/payments/admin/list	Admin	Retrieve the payment ledger
POST	/api/payments/admin/:id/confirm-upi	Admin	Confirm a reviewed transfer
GET	/api/feedback/event/:eventId	Public	Reviews and aggregate rating

Method	Endpoint	Access	Purpose
GET	/api/feedback/mine/:eventId	User	The caller's own review and eligibility
POST	/api/feedback	User	Submit a review
PUT	/api/feedback/:id	User	Edit the caller's review
DELETE	/api/feedback/:id	User	Withdraw the caller's review
GET	/api/feedback/admin/list	Admin	List every review
DELETE	/api/feedback/admin/:id	Admin	Remove any review
GET	/api/health	Public	Report API and database status

5.1.4 Request Lifecycle

A single authenticated request illustrates how the pieces fit together. A user submits the booking form. The page calls a function in `services/bookings.js`, which issues an Axios request through the shared client; the client's request interceptor attaches the stored token as an Authorization header. The request reaches Express, passes through CORS negotiation, JSON body parsing and request logging, and matches a route in `bookingRoutes.js`. That route names `protect`, which verifies the token signature and loads the account, and then `authorize('user')`, which checks the role. Only then does the controller run: it validates the quantity, applies the atomic conditional update that reserves the tickets, creates the booking, records the payment, and responds. Any error thrown along the way — including from a rejected promise — reaches the centralised error middleware, which formats it into a consistent JSON response. Back in the browser, the page updates from the response.

5.2 Coding Details and Code Efficiency

This section presents the code behind the mechanisms described above. Listings are given for the parts of the system where the implementation is not obvious from the design.

5.2.1 Password Hashing

The hashing hook is attached to the User schema rather than invoked from a controller. This is the decision that makes the guarantee absolute: any code path that saves a user document — a controller, a script, a future feature — passes through it.

```

userSchema.pre('save', async function savePassword(next) {
  if (!this.isModified('password')) {
    next();
    return;
  }

  const salt = await bcrypt.genSalt(10);
  this.password = await bcrypt.hash(this.password, salt);
  next();
});

userSchema.methods.comparePassword = async function
comparePassword(candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};

```

Listing 5.1: backend/models/User.js — password hashing and verification

The `isModified` guard matters: without it, every save of an existing user — a profile image update, for instance — would re-hash the already-hashed value and invalidate the password.

5.2.2 Token Issue and Verification

A token is signed with the account identifier and role, and verified on every protected request by middleware that runs before any controller.

```

const generateToken = (userId, role) =>
  jwt.sign({ userId, role }, process.env.JWT_SECRET, {
    expiresIn: process.env.JWT_EXPIRES_IN || '7d'
  });

```

Listing 5.2: backend/utils/generateToken.js

```

export const protect = async (req, res, next) => {
  const authHeader = req.headers.authorization;

  if (!authHeader || !authHeader.startsWith('Bearer ')) {
    res.status(401);
    next(new Error('Access denied. Token is missing or invalid.'));
    return;
  }

  try {
    const token = authHeader.split(' ')[1];
    const decoded = jwt.verify(token, process.env.JWT_SECRET);
    const user = await User.findById(decoded.userId).select('-password');

    if (!user) {
      res.status(401);
      next(new Error('User linked to this token no longer exists.'));
      return;
    }

    req.user = user;
    next();
  } catch (error) {
    res.status(401);
    next(new Error('Unauthorized access. Please log in again.'));
  }
}

```

```

    }
  };

  export const authorize = (...roles) => (req, res, next) => {
    if (!req.user || !roles.includes(req.user.role)) {
      res.status(403);
      next(new Error('You are not authorized to access this resource.'));
      return;
    }

    next();
  };

```

Listing 5.3: backend/middleware/authMiddleware.js — authentication and authorisation

Two details are deliberate. The account is re-loaded from the database on every request rather than trusted from the token payload, so a deleted or role-changed account is caught immediately instead of remaining valid until the token expires. And `authorize()` is a factory rather than a fixed guard, so a route declares the roles it accepts inline. The composition reads as the route's own security statement:

```

router.get('/admin/list', protect, authorize('admin'), getAdminBookings);
router.post('/admin/:id/transfer', protect, authorize('admin'),
  transferAdminBooking);
router.get('/', protect, authorize('user'), getMyBookings);

```

Listing 5.4: backend/routes/bookingRoutes.js — middleware composition

5.2.3 Ticket Inventory Conservation

This is the central mechanism of the system. The check on availability and the decrement of the count are expressed as one conditional update rather than as a read followed by a write. The query matches the event only if it is published and holds at least the requested quantity; if the match succeeds, the decrement is applied in the same operation. Because MongoDB applies a document update atomically, two simultaneous requests for the last remaining ticket cannot both match.

```

const ensureBookableEvent = async (eventId, quantity) => {
  let event = await Event.findOneAndUpdate(
    {
      _id: eventId,
      status: 'published',
      availableTickets: { $gte: quantity }
    },
    {
      $inc: { availableTickets: -quantity }
    },
    {
      new: true
    }
  );

  if (!event) {
    return null;
  }

  if (event.availableTickets === 0 && event.status !== 'sold_out') {
    event.status = 'sold_out';
    await event.save();
  }

  return event;
};

```

Listing 5.5: backend/controllers/bookingController.js — atomic ticket reservation

The inverse operation must restore both halves of what the booking consumed — the numeric count and the status transition the count triggered. Omitting the second was the defect recorded as TC-22 in Section 6.1.2.

```

const restoreInventoryForBooking = async (booking) => {
  const event = await Event.findById(booking.event);

  if (!event) {
    return;
  }

  event.availableTickets += booking.quantity;

  if (event.status === 'sold_out') {
    event.status = 'published';
  }

  await event.save();
};

```

Listing 5.6: backend/controllers/bookingController.js — inventory restoration

5.2.4 The Payment Ledger

Every payment attempt, whatever channel produced it, is written through one helper. The provider-to-method mapping lives in a single table, so the two payment controllers cannot derive it differently.

```

const methodByProvider = {
  demo: 'demo_gateway',
  upi_manual: 'upi_transfer',
  manual: 'offline',
  free: 'none'
};

export const recordPayment = async ({
  booking, event, userId, provider, amount, status, transactionId, confirmedBy
}) =>
  Payment.create({
    paymentReference: generatePaymentReference(),
    user: userId,
    event: event._id,
    booking: booking._id,
    bookingReference: booking.bookingReference,
    eventTitle: event.title,
    provider,
    method: methodByProvider[provider] || 'none',
    amount,
    status,
    transactionId,
    confirmedBy,
    confirmedAt: status === 'paid' ? new Date() : undefined
  });

```

Listing 5.7: backend/utils/paymentLedger.js — recording a payment attempt

The corresponding reversal is a bulk update rather than a per-document loop, so a cancellation touches the ledger once regardless of how many entries a booking has accumulated:

```

export const markPaymentsRefunded = async (bookingId) =>
  Payment.updateMany(
    { booking: bookingId, status: { $in: ['pending', 'paid'] } },
    { $set: { status: 'refunded', refundedAt: new Date() } }
  );

```

Listing 5.8: backend/utils/paymentLedger.js — reversing a payment on cancellation

5.2.5 Slug Generation

An event is addressed publicly by a slug derived from its title. Because two events may legitimately share a title, derivation alone is insufficient — the slug must be tested for collision and disambiguated. This was added in response to the defect recorded as TC-16.

```

const createSlug = (title) =>
  title
    .toLowerCase()
    .trim()
    .replace(/^[^a-z0-9]+/g, '-')
    .replace(/^-+|-+$/g, '');

const ensureUniqueSlug = async (title, excludeId) => {
  const baseSlug = createSlug(title);
  let slug = baseSlug || `event-${Date.now()}`;
  let suffix = 1;

  while (true) {
    const existingEvent = await Event.findOne({
      slug,
      ...(excludeId ? { _id: { $ne: excludeId } } : {})
    });

    if (!existingEvent) {
      return slug;
    }

    slug = `${baseSlug}-${suffix}`;
    suffix += 1;
  }
};

```

Listing 5.9: backend/controllers/eventController.js — unique slug derivation

5.2.6 Token Attachment on the Client

The access token is attached by a single Axios request interceptor, so no individual page has to remember to authenticate its own calls.

```

const api = axios.create({
  baseURL: import.meta.env.VITE_API_BASE_URL || '/api',
  timeout: 10000,
});

export const setApiAuthToken = (token) => {
  authToken = token;
};

api.interceptors.request.use((config) => {
  if (authToken) {
    config.headers.Authorization = `Bearer ${authToken}`;
  }
  return config;
});

```

Listing 5.10: src/services/api.js — the shared HTTP client

5.2.7 On-Demand Loading of the Ticket Libraries

The QR encoder is fetched at the moment a ticket is rendered rather than bundled into the initial payload, using a dynamic import. The same treatment is applied to the canvas rasteriser and the PDF generator.

```

useEffect(() => {
  let isMounted = true;

  const generateCode = async () => {
    try {
      const { default: QRCode } = await import('qrcode');
      const nextQrCodeSrc = await QRCode.toDataURL(qrPayload, {
        errorCorrectionLevel: 'M',
        margin: 1,
        width: 192
      });

      if (isMounted) {
        setQrCodeSrc(nextQrCodeSrc);
      }
    } catch (error) {
      if (isMounted) {
        setQrCodeSrc('');
      }
    }
  };

  generateCode();

  return () => {
    isMounted = false;
  };
}, [qrPayload]);

```

Listing 5.11: src/components/BookingTicket.jsx — deferred QR generation

The `isMounted` flag and the cleanup function prevent a state update on a component that has already been unmounted, which would otherwise occur if the user navigated away while the encoder was still loading.

5.2.8 Centralised Error Handling

Every failure in the API is formatted by one middleware, so no controller composes its own error response and no internal detail escapes in production.

```

const errorMiddleware = (error, req, res, next) => {
  const statusCode = res.statusCode && res.statusCode !== 200 ? res.statusCode : 500;

  res.status(statusCode).json({
    success: false,
    message: error.message || 'Internal server error.',
    stack: process.env.NODE_ENV === 'production' ? undefined : error.stack
  });
};

```

Listing 5.12: backend/middleware/errorMiddleware.js

5.2.9 Code Efficiency

Efficiency was addressed in five specific ways.

Denormalisation to eliminate query fan-out. Booking and Payment documents store a snapshot of the event fields needed for display. Rendering a user’s booking history therefore requires one query against bookings, not one query plus a populate call per booking. On a history of twenty bookings this is the difference between one round trip and twenty-one.

Targeted indexes. The Payment schema declares indexes on {booking, createdAt}, {user, createdAt} and {status} — the three access patterns the application actually uses — and the Feedback schema declares the compound unique index that enforces one review per user per event. Indexes were added for observed queries rather than speculatively, since each one costs write throughput and storage.

Field projection. Queries select only the fields they need. The authentication middleware uses .select('-password') so the hash is never even loaded into memory, and the booking controller loads only price when all it needs to know is whether an event is free.

On-demand code loading. The QR code, canvas rasterisation and PDF libraries are loaded through dynamic import() at the moment a ticket is requested rather than being bundled into the initial payload. These three libraries together are among the largest dependencies in the project, and the majority of page views never need them.

Single-operation reservation. Booking creation performs its availability check and its inventory decrement as one conditional database update rather than as separate read and write round trips. This is both faster and correct under concurrency, since there is no window between the two in which a competing request could intervene.

5.3 Testing Approach

Testing was carried out continuously rather than as a phase at the end. The strategy proceeded from the smallest units outward: individual functions and endpoints first, then the interactions between modules, then the complete system in its deployed environment. This ordering is deliberate — a defect found in an isolated unit has exactly one possible cause, whereas the same defect surfacing in an end-to-end flow could originate anywhere along it.

5.3.1 Unit Testing

Unit testing examined individual functions and endpoints in isolation, with each API route exercised through Postman independently of any interface.

For the authentication module this meant testing registration against valid input, a missing field, a malformed email address, a password shorter than the minimum, a password that did not match its confirmation, and an email already registered. Sign-in was tested with correct credentials, an incorrect password, an unregistered email, and a missing field. Token verification was tested with a valid token, no token, a malformed token, a token signed with the wrong secret, and a token whose subject had been deleted. Role authorisation was tested by presenting a user token to an administrative endpoint, and an ordinary account to the administrative sign-in route.

For the event module, creation was tested against valid input and against each boundary the schema enforces: zero tickets, a negative price, an over-length title, an end date preceding the start date, a missing required field, and an image exceeding the permitted size. Slug generation was tested by creating two events with identical titles and confirming that distinct slugs resulted.

For the booking module the tests centred on the availability boundary: a quantity below availability, exactly equal to availability, and one above it. The exact-availability case is the important one, since it is where the sold-out transition fires.

For the payment module each channel was tested separately — free, demo gateway, and manual UPI — along with an under-length transfer reference and the double-confirmation case, in which an already-confirmed payment is confirmed a second time. For the feedback module, submission was tested with and without a confirmed booking, with an out-of-range rating, with an over-length comment, and twice for the same event to confirm the uniqueness constraint.

The value of testing at this level was that failures were unambiguous. When the over-availability booking initially succeeded, the fault could only lie in the availability check itself, because nothing else was involved.

5.3.2 Integration Testing

Integration testing verified that the modules behave correctly in combination, and that data crossing a boundary between them arrives intact.

The frontend-to-backend integration was tested by submitting each form through the interface and confirming that the request reached the correct endpoint with the expected payload and headers, and that the response was rendered correctly. The token interceptor was verified by confirming that a request issued after sign-in carried the Authorization header, and that one issued after sign-out did not.

The backend-to-database integration was tested by confirming that each write produced the expected document, that schema validators rejected invalid writes, and that the unique index on email rejected a duplicate registration even when the application-level check was bypassed by issuing the request directly.

The cross-module integrations were the ones that mattered most. The booking-to-event integration was verified by booking tickets and confirming that the event's available count decreased by exactly the quantity booked — and, on cancellation, increased by exactly the same amount and restored the published status. The booking-to-payment integration was verified by confirming that each booking produced exactly one payment record with the correct amount, provider and status, and that cancelling a paid booking marked its payment refunded. The booking-to-feedback integration was verified by confirming that a review is accepted only from an account holding a confirmed booking for that event. The administrative UPI confirmation flow was verified end to end: a customer submits a transfer reference, the payment is recorded as pending and the booking held, an administrator confirms it, and the booking moves to confirmed with the confirming administrator and timestamp recorded.

5.3.3 System Testing

System testing exercised the complete application in its deployed environment — the frontend on Vercel, the backend on Railway and the database on MongoDB Atlas — rather than against local servers, so that deployment-specific behaviour was covered.

Two complete journeys were traced. In the user journey, a new visitor registers, signs in, browses the events listing, opens an event, books tickets, completes payment through each of the three channels in turn, views the booking in their history, downloads the QR-coded ticket as a PDF, submits a review, cancels a booking and confirms that the tickets return to the event. In the administrator journey, an administrator signs in through the hidden route, reviews the full event listing, features an event and confirms it appears on the landing page, reviews the booking list, confirms a pending UPI payment, verifies the

ticket activates, transfers a booking to another account, moderates a review, and reviews the payment ledger and the user list.

Alongside these, cross-cutting behaviour was checked. Every protected route was requested without a token and with a token of the wrong role, and each was confirmed to reject the request. The interface was exercised at desktop, tablet and mobile viewport widths in both themes. The single-page routing was verified by deep-linking to and refreshing an interior route, which confirmed that the Vercel rewrite was serving `index.html` correctly. Cross-origin behaviour was verified by confirming that the API accepts requests from the deployed frontend origin and rejects them from another. The health endpoint was queried to confirm that it reports database connectivity accurately.

5.4 Modifications and Improvements

The following changes were made during development in response to problems found in testing or to weaknesses identified on review. They are presented in the order they occurred, which follows the project's version control history.

No.	Issue identified	Modification made
1	A page refresh on an interior route returned a 404 from the static host.	A rewrite rule was added in <code>vercel.json</code> directing all paths to <code>index.html</code> , so that client-side routing resolves the path after the shell loads.
2	Cancelling a booking on a sold-out event returned the tickets, but the event remained unbookable.	<code>restoreInventoryForBooking</code> was extended to reset the event's status from <code>sold_out</code> back to published whenever stock is returned.
3	Editing an event's venue or start time silently altered the displayed details of every past booking against it.	Booking and Payment schemas were given denormalised snapshot fields, captured at creation, so a historical record reflects the event as it stood at purchase.
4	The interface was usable on desktop but cramped on narrow viewports.	Layouts were reworked to Bootstrap's mobile-first breakpoints; listings collapse to a single column and administrative tables scroll within their container.
5	A single payment path was insufficient, since no live gateway was available.	A demo gateway and a manually-reviewed UPI channel were implemented side by side, both writing to a shared payment ledger.
6	Payment method strings were being derived independently in each payment controller, allowing them to drift.	The <code>methodByProvider</code> mapping and the <code>recordPayment</code> helper were extracted so that every channel writes ledger entries through one code path.
7	An administrative sign-in link in the public navigation advertised the existence of the administrative area.	The link was removed and the route made reachable through a hidden trigger — five rapid clicks on the footer text, or a keyboard shortcut.
8	Protected pages briefly redirected legitimate users to the sign-in screen while the stored session was being restored.	<code>ProtectedRoute</code> was given an explicit loading state, so a session still being restored renders a placeholder rather than being treated as unauthenticated.
9	Bundling the QR, canvas and PDF libraries made the initial payload large for every	The three libraries were moved behind dynamic <code>import()</code> calls executed at the moment a ticket is

No.	Issue identified	Modification made
	visitor, including those who never download a ticket.	requested.
10	Administrators could see bookings but could not act on them when a customer needed assistance.	Administrative endpoints were added to create, cancel and transfer a booking on a user's behalf.
11	Featured events had to be unfeatured by hand once they had passed.	A featuredUntil expiry date was added alongside the featured flag, bounded to a window of one to twenty-four hours.
12	There was no way to confirm from outside whether the API had reached the database.	A /api/health endpoint was added, reporting API and database status, and the connection module was reworked to track and expose its own state.
13	A booking could in principle be created between the availability check and the inventory decrement, allowing an oversell under concurrent load.	The check and the decrement were merged into a single conditional findOneAndUpdate, making the reservation atomic.
14	Attendees had no way to record their experience of an event, and organisers had no structured feedback.	A feedback module was implemented, restricted to holders of a confirmed booking, with a compound unique index enforcing one review per user per event and an administrative moderation console.

5.4.1 Outstanding Items

Two items identified during review remain outstanding and are recorded here for completeness. A diagnostic console.log of the configured API base URL remains in src/services/api.js and should be removed before any production release. A PaymentOrder model referring to a Razorpay order identifier remains in backend/models/ from an abandoned attempt at commercial gateway integration; it is referenced by no route or controller and should be deleted. Neither affects the behaviour of the system, but both are the kind of residue that a reviewer will notice.

RESULTS AND DISCUSSION

6.1 Test Reports

6.1.1 Summary of Results

The forty-eight test cases designed in Section 4.5 were executed against the deployed system. The table below summarises the outcome by module.

Module	Cases	Passed	Failed	Pass rate
User Management and Authentication	11	11	0	100%
Event Management	9	9	0	100%
Booking Management	8	8	0	100%
Payment Management	7	7	0	100%
Ticketing	2	2	0	100%
Feedback Management	4	4	0	100%
Administration	6	6	0	100%
System	1	1	0	100%
Total	48	48	0	100%

The figures above record the state of the system after the modifications described in Section 5.4. Several of these cases did not pass on first execution; the defects they exposed are the subject of the next section, and it is those results rather than the final totals that carry the useful information.

6.1.2 Defects Found and Resolved

Case	Observed behaviour	Cause	Resolution
TC-22	After a booking consumed the last available tickets, the event was marked sold out — but cancelling that booking returned the tickets and left the event still marked sold out and therefore unbookable.	The cancellation path restored the numeric count but did not reverse the status transition that the count had triggered.	The status reset was added to <code>restoreInventoryForBooking</code> , so any return of stock republishes a sold-out event.
TC-27	Cancelling a paid booking returned the tickets but left the associated payment recorded as paid.	The cancellation path updated the booking and the event but not the ledger.	<code>markPaymentsRefunded</code> was called as part of the cancellation, so the ledger reflects the reversal.
TC-19	Refreshing the browser on an event detail page returned a 404 from the static host.	The host had no rewrite rule, so it attempted to resolve the client-side route as a file path.	A catch-all rewrite to <code>index.html</code> was added in <code>vercel.json</code> .
TC-06	A signed-in user reloading a protected page was briefly redirected to the sign-in screen before being returned.	The route guard treated the interval during which the stored session was being restored as an unauthenticated state.	An explicit loading branch was added to <code>ProtectedRoute</code> .

Case	Observed behaviour	Cause	Resolution
TC-34	An already-confirmed UPI payment could be confirmed a second time, writing a second confirmation timestamp.	The confirmation handler did not check the payment's current status before acting.	A status guard was added, rejecting confirmation of a booking not awaiting UPI review.
TC-16	Two events created with the same title produced a duplicate-key error on the slug index.	Slug derivation was a direct transformation of the title with no uniqueness check.	Slug generation was extended to test for collision and disambiguate with a numeric suffix.
TC-23	Under rapid repeated requests, a booking could occasionally be accepted for more tickets than remained.	The availability check and the inventory decrement were separate operations, leaving a window in which a competing request could intervene.	The two were merged into a single conditional <code>findOneAndUpdate</code> , making the reservation atomic.
TC-40	A user could submit two reviews for the same event by issuing the request twice in quick succession.	The duplicate check was performed in the controller only, which two concurrent requests could both pass.	A compound unique index on {user, event} was added to the Feedback schema, enforcing the constraint at the database level.

6.1.3 Discussion

Three observations are worth drawing out of these results.

The defects clustered around state transitions, not around individual operations.

Every operation worked correctly when tested on its own. What failed was the correspondence between operations — a status set by one action and not cleared by its inverse, a ledger updated on payment but not on refund, a confirmation that could be applied twice. This is the characteristic failure mode of a system holding related state in several places, and it is precisely why the availability invariant described in Section 4.2.2 was made explicit: once the rule is stated as “every decrement has a matching restore”, the omissions become findable rather than a matter of luck.

Boundary cases were where the useful defects lived. Booking fewer tickets than were available worked from the beginning. Booking exactly the number available is what exposed both the sold-out reversal defect and, indirectly, the ledger defect. The general lesson — that the interesting case is the one at the edge of a condition rather than comfortably inside it — guided the later test design, and cases TC-22, TC-23 and TC-34 were all written specifically to sit on such boundaries.

A constraint the application checks is not a constraint the system enforces. Two of the defects above — the oversell under contention and the duplicate review — had the same shape: a rule correctly stated in a controller, and correctly enforced for any single request, that two concurrent requests could both pass. In each case the fix was to move the guarantee down a layer, into an atomic database operation or a unique index, where

concurrency cannot defeat it. This is the practical difference between validating input and enforcing an invariant.

The system's behaviour under the conditions tested is therefore sound, with the qualification that testing was functional rather than load-based. No formal load or stress testing was performed, so while the reservation path is now atomic by construction, its throughput under sustained concurrent pressure has not been measured. This is recorded as a limitation in Section 7.2.

6.2 User Documentation

This section describes how to use the completed application. It is written for the two roles the system recognises, and assumes no technical knowledge.

6.2.1 Getting Started

The application is reached through a web browser at the deployed address and requires no installation. Visitors may browse published events and view their details without an account; registering is required only to book tickets, publish an event or leave a review. A theme toggle in the navigation bar switches the interface between light and dark presentation, and the choice is remembered.

6.2.2 For Registered Users

6.2.2.1 Creating an Account

Select Sign up from the navigation bar. Supply a name, an email address and a password of at least six characters, and confirm the password. The email address must not already be registered. On successful registration the account is created and signed in immediately.

6.2.2.2 Signing In

Select Sign in and supply the registered email address and password. On success the user dashboard is displayed. An incorrect email or password produces a message to that effect without indicating which of the two was wrong.

6.2.2.3 Browsing and Finding Events

The landing page presents a selection of featured events. The Events page lists every published event. Selecting an event opens its detail page, which shows the full

description, category, organiser, venue and city, start and end date-times, ticket price, the number of tickets still available, and the reviews left by previous attendees with their aggregate rating.

6.2.2.4 Booking Tickets

On an event page, enter the number of tickets required and select Book. The quantity must be a whole number and cannot exceed the tickets still available. A free event is confirmed immediately. A priced event proceeds to payment.

6.2.2.5 Paying for a Booking

Two channels are offered for a priced event. The demo gateway confirms the booking immediately and records the payment as paid. The manual UPI route displays the payment details; after completing the transfer in a UPI application, enter the transaction reference and submit it. The booking is then held pending administrative review, and the ticket activates once an administrator confirms the transfer.

6.2.2.6 Viewing Bookings and Downloading Tickets

The Bookings page lists every booking made by the account, most recent first, with its reference code, event, quantity, amount and current status. Each confirmed booking displays a ticket carrying a QR code. Select Download Ticket PDF to save a copy; the file matches what is shown on screen and can be presented at the venue.

6.2.2.7 Cancelling a Booking

Select Cancel on a booking and confirm. The booking is marked cancelled, the associated payment is marked refunded in the ledger, and the tickets are returned to the event — which becomes bookable again if it had sold out. A booking already cancelled cannot be cancelled a second time.

6.2.2.8 Leaving a Review

A user holding a confirmed booking for an event may award a star rating between one and five and write a review of up to 800 characters. The review appears on the event page and contributes to its aggregate rating. Each account may hold one review per event, which may be edited or withdrawn at any time.

6.2.2.9 Publishing an Event

From My Events, select Create Event and supply the title, description, category, organiser name, venue, city, start and end date-times, ticket price and total ticket count, optionally attaching a banner image. The event is published on saving and appears in the public listing. Events published by the account can be edited or deleted from the same page.

6.2.2.10 Profile and Signing Out

The account menu allows a profile photograph to be uploaded — a PNG, JPEG or WebP image under one megabyte — and the interface theme to be changed. Select Sign out to end the session; the stored token is discarded and the public view is restored.

6.2.3 For Administrators

6.2.3.1 Reaching the Administrative Sign-In

The administrative sign-in page is not linked from the public navigation. It is reached either by clicking the footer text on the left five times in rapid succession, or by pressing Ctrl+Shift+L on any page. Supply the administrator credentials configured for the deployment. An ordinary account presented here is rejected.

6.2.3.2 Managing Events

The administrative dashboard lists every event on the platform regardless of status, with its owner, dates, price and ticket counts. Any event may be edited or deleted. An event may be given featured placement on the landing page for a chosen window of between one and twenty-four hours, after which the placement lapses automatically; it may also be withdrawn manually before then.

6.2.3.3 Managing Bookings

The Bookings console lists every booking on the platform with its user, event, quantity, amount and status. An administrator may create a booking on behalf of a chosen user, cancel any booking — which restores the tickets and refunds the ledger entry — or transfer a booking to a different account, which is the route for correcting a booking made under the wrong name.

6.2.3.4 Managing Payments

The Payments console presents the complete ledger: every payment attempt with its reference, user, event, provider, method, amount, status and, where applicable, the administrator who confirmed it and when. A UPI transfer awaiting review is confirmed from here, which marks the payment paid and activates the ticket.

6.2.3.5 Moderating Feedback

The Feedback console lists every review submitted across the platform with its author, event, rating and comment. A review may be removed, after which the event's aggregate rating is recalculated without it.

6.2.3.6 Managing Users

The user listing shows every registered account with its name, email and registration date. Password hashes are never returned to the interface.

6.2.3.7 Checking System Health

Requesting `/api/health` returns the current status of the API together with the state of the database connection, which is the quickest way to establish whether a fault lies in the application or in its connection to the data tier.

6.2.4 Troubleshooting

Symptom	Explanation and remedy
"An account with this email already exists."	The address is already registered. Sign in instead, or register with a different address.
"Invalid email or password."	One of the two is incorrect. The message is deliberately unspecific so that it does not reveal whether an address is registered.
"Tickets are no longer available for this event."	The requested quantity exceeds what remains, or the last tickets were taken by another user first. Reload the event page to see the current availability.
"Please complete payment before confirming this booking."	A priced event cannot be booked directly; proceed through the demo gateway or the UPI route.
A UPI booking shows "Payment Review Pending".	The transfer reference has been submitted but not yet confirmed by an administrator. The ticket activates once it is.
"You are not authorized to access this resource."	The operation requires a role the signed-in account does not hold. Administrative operations require an administrator account.
Signed out unexpectedly.	The access token has expired. Sign in again.
The QR code shows "Generating QR".	The encoder is still loading. If it does not resolve, reload the page; the library is fetched on demand and a network interruption can interrupt it.

CONCLUSION

7.1 Conclusion

The objective set out at the beginning of this project was to replace the fragmented, manual administration of events with a single application in which an event, the tickets sold against it and the payments received for it form one connected record. That objective has been met. Eventify is a working, deployed, full-stack web application in which a registered user can publish an event, others can discover and book it, payment is settled through one of three channels and recorded in an auditable ledger, a QR-coded ticket is issued against every confirmed booking, attendees can review what they attended, and an administrator has oversight of the whole platform.

The technical foundation is a MERN stack: a React 18 single-page frontend bundled with Vite and styled with Bootstrap 5, an Express 5 REST API on Node.js, and a MongoDB Atlas cluster accessed through Mongoose. Authentication is stateless, with bcrypt-hashed passwords and signed JSON Web Tokens, and authorisation is enforced as composable route middleware so that role restrictions live in the route table rather than being scattered through controllers. The application was verified through forty-eight designed test cases across unit, integration and system levels, and deployed with the frontend on Vercel, the backend on Railway and the database on MongoDB Atlas.

Three design decisions proved to be the ones that mattered.

Treating ticket inventory as a conserved quantity converted a class of administrative error into a structural impossibility. Because a single atomic conditional update both validates availability and decrements it, and every cancellation restores it, the availability shown to a prospective attendee cannot drift from reality. Overselling is not discouraged by the system; it is prevented by it.

Denormalising the event snapshot into bookings and payments solved a problem that was not obvious at the outset. A booking is a record of a transaction that occurred at a particular moment, and if it resolves the event by reference at read time, a later edit to that event silently rewrites history. Storing the title, slug, start date, venue and city as they stood at purchase costs a small amount of duplication and buys a record that stays true.

Enforcing authorisation at the API rather than in the interface is the difference between a system that appears secure and one that is. Hiding an administrative link protects nothing; the route behind it is protected independently, and the interface guard exists for usability alone.

The defects surfaced during testing are also instructive, and they clustered in one place: not in individual operations, all of which worked in isolation, but in the correspondence between an operation and its inverse — a sold-out status set on booking and not cleared on cancellation, a ledger entry written on payment and not reversed on refund, a confirmation that could be applied twice — and in rules that a controller checked but only a database constraint could actually enforce. Both are characteristic failure modes of a system holding related state in more than one place, and identifying them explicitly made the remaining instances findable rather than a matter of chance.

Taken together, the project demonstrates that the operational problems of manual event administration — duplicated records, uncertain availability and untraceable payments — are addressable with a modest, carefully structured full-stack application, and it provides a foundation on which the extensions described in Section 7.3 can be built.

7.1.1 Significance of the System

The significance of the system lies in three areas.

Operationally, it removes the manual reconciliation that consumes most of the effort in running an event. Because registration, ticket count and payment are parts of one record rather than three artefacts kept in step by hand, the reconciliation step disappears entirely rather than merely becoming easier. For an organiser this is the difference between an administrative task and no task at all.

Structurally, it distributes event publication rather than centralising it. Because any registered user may publish an event and administrators moderate rather than originate, the platform suits a setting — a college in which individual societies each run their own programme, for instance — where a single administrative office would be a bottleneck. This is a deliberate departure from the conventional administrator-publishes model, and it changes what the platform is useful for.

Academically, the project demonstrates the practical construction of a complete full-stack application: component-based frontend architecture, RESTful API design with

composed middleware, schema modelling on a document database, stateless authentication with role-based authorisation, and independent deployment of the presentation and application tiers. The problems encountered — state transitions that were not reversed, records that changed retroactively, constraints that concurrency defeated — are the problems that characterise real systems rather than exercises, and working through them was the more valuable part of the exercise.

7.2 Limitations of the System

The system as delivered has the following limitations. They are stated plainly, since a clear account of what a system does not do is more useful than an optimistic one.

Payment is not settled through a live gateway. The demo gateway simulates settlement, and the UPI channel depends on an administrator manually verifying that a transfer was received. Neither produces the automatic, cryptographically verified confirmation that a commercial gateway provides, and the UPI channel introduces a human step between payment and ticket activation.

Ticket QR codes are issued but not validated. Every confirmed booking yields a scannable QR code, but no companion application exists to scan it at a venue entrance and mark it used. A ticket could therefore be presented more than once without the system detecting it.

No notification of any kind is sent. There is no email or SMS confirmation on registration, on booking, on payment, or when an event is changed or cancelled. A user learns of a change only by returning to the site.

Throughput under sustained load has not been measured. The reservation path is atomic by construction, so an oversell cannot occur under contention, but no formal load or stress testing was performed. The system's behaviour under sustained concurrent pressure — response times, connection pool saturation — has not been established empirically.

Tokens cannot be revoked. Because authentication is stateless, a token remains valid until it expires. There is no mechanism to invalidate a token early, so a compromised token stays usable for the remainder of its lifetime.

Authentication endpoints are not rate limited. The system does not itself slow repeated sign-in attempts, although bcrypt's cost factor makes automated guessing expensive.

Administrative insight is limited to listings. The consoles show every event, booking, payment, review and user, but provide no aggregation — no revenue totals, no sales trends, no measure of event performance.

Search and filtering are basic. Events can be browsed and filtered by category, but not by city, date range or price, and there is no full-text search. This becomes a limitation as the number of listings grows.

Seating is not modelled. Tickets are counted, not assigned. Events at venues with numbered seats cannot be represented.

Banner images are stored inside event documents as base64 strings rather than in object storage, which inflates document size and is not an approach that scales to a large catalogue of events.

There is no native mobile application. The interface is responsive and works in a mobile browser, but offers none of the capabilities — offline ticket access, push notification — that a native application would.

7.3 Future Scope of the Project

Each of the following extends the system in a direction the current architecture already accommodates.

7.3.1 Live Payment Gateway Integration

Replacing the demo gateway with a commercial provider such as Razorpay or Stripe would give automatic, verified settlement and remove the manual confirmation step from the UPI path. The groundwork exists: the payment ledger already records a provider, a method and a transaction identifier for every attempt, so a new provider is an additional entry in the existing enumeration rather than a restructuring. Webhook handling would be required so that settlement confirmation arrives from the provider rather than from the client.

7.3.2 Ticket Validation at the Venue

A companion scanning application — a web application usable from any phone camera would suffice — would read a ticket's QR code, verify the booking against the API and mark it as used. This closes the gap identified in Section 7.2: a ticket presented a second

time would be rejected. It requires a scanned-at timestamp on the booking and one additional endpoint.

7.3.3 Notification

Transactional email and SMS on registration, booking, payment confirmation, event modification and event cancellation, together with a reminder before an event begins. Since the system already records every state transition, notification is a matter of dispatching a message at each transition rather than of tracking new state.

7.3.4 Token Revocation and Rate Limiting

Introducing refresh tokens with short-lived access tokens, backed by a revocation list, would allow a compromised session to be terminated without abandoning the benefits of stateless authentication. Rate limiting on the authentication endpoints would slow automated credential guessing at the perimeter.

7.3.5 Search, Filtering and Recommendation

Filtering by city, date range, price and availability, with full-text search over titles and descriptions — both supported natively by MongoDB. Beyond that, a recommendation mechanism suggesting events based on a user's booking history and the categories they browse.

7.3.6 Analytics and Reporting

An administrative dashboard aggregating ticket sales, revenue, booking trends over time, event popularity by category and city, average ratings and cancellation rates, with export to spreadsheet for offline analysis. MongoDB's aggregation pipeline supports this directly.

7.3.7 Seat Selection

A visual seat map for venues with assigned seating, allowing a customer to choose specific seats before confirming. This requires extending the event model with a seating layout and the booking model with the specific seats reserved.

7.3.8 Media Storage

Moving banner images out of event documents into an object store, with the document retaining only a URL. This reduces document size, allows images to be served from a content delivery network, and removes a scaling constraint.

7.3.9 Load and Performance Testing

Formal load testing to establish throughput, response-time distribution and connection pool behaviour under sustained concurrent use, which would convert the correctness guarantee already in place into a capacity figure.

7.3.10 Native Mobile Applications

Android and iOS applications consuming the same REST API, offering offline access to purchased tickets and push notification — the two capabilities a responsive web interface cannot provide.

7.3.11 Multi-Language Support

Interface localisation, which broadens the platform's reach in a multilingual setting.

These extensions are directions in which the system could develop rather than commitments within the scope of the present academic project. What matters for the work described in this report is that none of them requires the existing architecture to be dismantled: each is an addition to a structure that was designed to accommodate it.

REFERENCES

The following sources were consulted during the design, implementation and testing of the Smart Event Management and Ticket Booking System.

Official Documentation

- [1] Meta Open Source. React Documentation — Learn React. Available at: <https://react.dev/learn>
- [2] Meta Open Source. React Documentation — Passing Data Deeply with Context. Available at: <https://react.dev/learn/passing-data-deeply-with-context>
- [3] Vite. Vite Guide — Getting Started. Available at: <https://vite.dev/guide/>
- [4] Vite. Vite Guide — Building for Production. Available at: <https://vite.dev/guide/build>
- [5] Remix Software. React Router Documentation. Available at: <https://reactrouter.com/>
- [6] OpenJS Foundation. Node.js Documentation — API Reference. Available at: <https://nodejs.org/docs/latest/api/>
- [7] OpenJS Foundation. Express 5.x — API Reference. Available at: <https://expressjs.com/en/5x/api.html>
- [8] OpenJS Foundation. Express — Using Middleware. Available at: <https://expressjs.com/en/guide/using-middleware.html>
- [9] MongoDB, Inc. MongoDB Manual — Data Modeling. Available at: <https://www.mongodb.com/docs/manual/data-modeling/>
- [10] MongoDB, Inc. MongoDB Manual — Indexes. Available at: <https://www.mongodb.com/docs/manual/indexes/>
- [11] MongoDB, Inc. MongoDB Atlas Documentation. Available at: <https://www.mongodb.com/docs/atlas/>
- [12] Automattic. Mongoose Documentation — Guide. Available at: <https://mongoosejs.com/docs/guide.html>
- [13] Automattic. Mongoose Documentation — Validation. Available at: <https://mongoosejs.com/docs/validation.html>
- [14] Automattic. Mongoose Documentation — Middleware. Available at: <https://mongoosejs.com/docs/middleware.html>

- [15] Bootstrap Team. Bootstrap 5 Documentation — Layout and Grid System. Available at: <https://getbootstrap.com/docs/5.3/layout/grid/>
- [16] Axios. Axios Documentation — Interceptors. Available at: <https://axios-http.com/docs/interceptors>
- [17] MDN Web Docs. HTTP — Cross-Origin Resource Sharing (CORS). Available at: <https://developer.mozilla.org/en-US/docs/Web/HTTP/CORS>
- [18] MDN Web Docs. JavaScript — Modules and Dynamic Imports. Available at: <https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Modules>
- [19] MDN Web Docs. CSS — Using CSS Custom Properties. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/Using_CSS_custom_properties
- [20] Vercel. Vercel Documentation — Project Configuration. Available at: <https://vercel.com/docs/project-configuration>
- [21] Railway. Railway Documentation — Deploying a Node.js Application. Available at: <https://docs.railway.com/>
- [22] Git. Pro Git Book, 2nd edition. Available at: <https://git-scm.com/book/en/v2>
- [23] Postman. Postman Learning Center — Testing an API. Available at: <https://learning.postman.com/docs/>
- [24] MKLab. StarUML Documentation. Available at: <https://docs.staruml.io/>

Standards and Specifications

- [25] Jones, M., Bradley, J. and Sakimura, N. RFC 7519: JSON Web Token (JWT). Internet Engineering Task Force, 2015. Available at: <https://datatracker.ietf.org/doc/html/rfc7519>
- [26] Fielding, R. and Reschke, J. RFC 7231: Hypertext Transfer Protocol (HTTP/1.1) — Semantics and Content. Internet Engineering Task Force, 2014. Available at: <https://datatracker.ietf.org/doc/html/rfc7231>
- [27] Provos, N. and Mazières, D. A Future-Adaptable Password Scheme. Proceedings of the USENIX Annual Technical Conference, 1999.
- [28] Object Management Group. OMG Unified Modeling Language (OMG UML), Version 2.5.1. Available at: <https://www.omg.org/spec/UML/>

[29] International Organization for Standardization. ISO/IEC 18004: Information technology — Automatic identification and data capture techniques — QR Code bar code symbology specification.

Security Guidance

[30] OWASP Foundation. OWASP Password Storage Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

[31] OWASP Foundation. OWASP Authentication Cheat Sheet. Available at: https://cheatsheetseries.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html

[32] OWASP Foundation. OWASP Top Ten Web Application Security Risks. Available at: <https://owasp.org/www-project-top-ten/>

Books

[33] Flanagan, D. JavaScript: The Definitive Guide, 7th edition. O'Reilly Media, 2020.

[34] Chodorow, K. MongoDB: The Definitive Guide, 3rd edition. O'Reilly Media, 2019.

[35] Sommerville, I. Software Engineering, 10th edition. Pearson, 2015.

[36] Pressman, R. S. and Maxim, B. R. Software Engineering: A Practitioner's Approach, 8th edition. McGraw-Hill Education, 2014.

[37] Fowler, M. Patterns of Enterprise Application Architecture. Addison-Wesley, 2002.

GLOSSARY

Term	Definition
API	Application Programming Interface. The set of HTTP endpoints through which the frontend communicates with the server.
Atomic operation	A database operation that completes entirely or not at all, and that no concurrent operation can observe in a partial state.
Axios	The HTTP client library used by the frontend to call the API.
bcrypt	A deliberately slow, salted password hashing function used to store credentials securely.
Bootstrap	The responsive CSS framework providing the grid system and interface components.
Booking reference	A unique human-readable code generated for each booking, of the form BKG-YYYYMMDD-XXXXXX.
BSON	Binary JSON — the storage format MongoDB uses for documents.
Collection	The MongoDB equivalent of a relational table: a group of documents.
CORS	Cross-Origin Resource Sharing. The browser mechanism controlling which origins may call the API.
Denormalisation	Deliberately duplicating a field from one document into another to avoid a lookup, and to preserve a historical value.
Document	A single record in a MongoDB collection, structured as a set of field-value pairs.
Express	The Node.js web framework providing routing and the middleware pipeline.
Featured event	An event given temporary placement on the landing page, bounded by an expiry timestamp.
Hook (Mongoose)	A function that Mongoose runs automatically before or after a schema operation, such as the pre-save password hash.
Hook (React)	A function that lets a React component use state and lifecycle behaviour.
Index	A data structure maintained by the database to make a particular query pattern efficient.
JSON	JavaScript Object Notation. The data interchange format used between the tiers.
JWT	JSON Web Token. A signed, self-describing token carrying the caller's identity and role.
MERN	MongoDB, Express, React, Node.js — the stack on which this system is built.
Middleware	A function in the Express pipeline that inspects or modifies a request before it reaches a controller.
MongoDB Atlas	The managed cloud service hosting the database cluster.
Mongoose	The Object Data Modelling library that adds a schema layer above MongoDB.
ODM	Object Data Modelling. The document-database counterpart of an ORM.
ObjectId	MongoDB's 12-byte unique document identifier.
Payment ledger	The payments collection, holding an auditable record of every settlement attempt.
populate()	The Mongoose method that resolves a reference field into the referenced document.

Term	Definition
QR code	A two-dimensional barcode; here it encodes a booking's identifying details on the ticket.
RBAC	Role-Based Access Control. Restricting operations according to the caller's role.
React	The JavaScript library used to build the user interface.
REST	Representational State Transfer. The architectural style the API follows.
Salt	Random data mixed into a password before hashing so that identical passwords produce different hashes.
Schema	The Mongoose declaration of a collection's permitted fields, types and constraints.
Slug	A URL-friendly identifier derived from an event's title, used to address it publicly.
SPA	Single-Page Application. A web application that rewrites the current page rather than loading new ones from the server.
Token interceptor	The Axios hook that attaches the stored access token to every outgoing request.
UPI	Unified Payments Interface. The Indian real-time payment system used by the manual payment channel.
Vercel	The platform hosting the deployed frontend.
Railway	The platform hosting the deployed backend.
Vite	The build tool and development server used for the frontend.